

Contents

Appendix A Optimized Link State Routing (OLSR) Protocol ... 1

A.1	RFC series status and where RFC 3626 sits	1
A.2	Core Principles of OLSR	2
A.3	OLSR Packet Structure and Integrity	3
A.4	Core and Auxiliary Control Messages	5
A.5	Protocol Constants and Timers.....	10
A.6	Routing Table Construction	11
A.7	The Distinction Between Control and Data Planes	11

Appendix B Virtual Network Layer Implementation Details. 12

B.1	OlsrNode Class: The Complete Technical Blueprint.....	13
B.2	Detailed Packet and Message Structures	25
B.2.1	Message Body Structures	25
B.2.2	Construction of the Message Header	30
B.2.3	Construction of the Packet Header	32
B.3	Packet Assembly Process	33
B.4	Packet Serialization and Transmission Specifications.....	36
B.4.1	Serialization and Encoding: From Dictionary to Bytes	36
B.4.2	Network Transmission: The <code>_broadcast</code> Method...	37
B.4.3	Reception and Deserialization: From Bytes to Dictionary	37
B.5	Implementation of Packet Integrity Mechanisms	39
B.5.1	Structural and Syntactic Validation	40

B.5.2	Logical and Protocol-Specific Validation	41
B.6	Implementation of Message Transmission Logic	46
B.6.1	HELLO Message Transmission (_hello_sender_thread).....	46
B.6.2	Topology Control (TC) Message Transmission (_tc_sender_thread)	47
B.6.3	HNA and MID Message Transmission (_hna_sender_thread, _mid_sender_thread).....	48
B.7	Implementation of the Receiver Thread and Dispatch Logic .	50
B.7.1	Socket Initialization and Main Loop.....	51
B.7.2	Stage 1: Structural Validation (Decoding and Parsing)	51
B.7.3	Stage 2 & 3: Priority Command Processing and Blacklist Validation	52
B.7.4	Stage 4: Message Disassembly and Dispatch	53
B.8	Implementation of Message Processing Functions	56
B.8.1	HELLO Message Processing (_process_hello_message).....	56
B.8.2	TC Message Processing (_process_tc_message) 60	
B.8.3	HNA and MID Message Processing	62
B.8.4	DATA Message Processing (_process_data_message).	63
B.8.5	COMMAND Message Processing (_process_command_message)	68
	Use Cases:	71

B.9	Implementation of Routing Table Calculation	72
B.9.1	The Routing Table Thread (_routing_table_thread)	73
B.9.2	The Core Calculation Function (_calculate_routing_table)	73
B.10	Implementation of the State Cleanup Mechanism.....	77
B.10.1	The Cleanup Thread (_cleanup_thread)	77
B.11	Implementation of the Logging Framework.....	83
B.11.1	Logger Setup (_setup_loggers).....	83
B.11.2	Usage Examples	84
B.11.3	Shutdown Hook for Traffic Summaries	86
B.12	Implementation of the External Traffic Generator	88
B.12.1	The test_runner.py Script	88
B.12.2	Interaction with OlsrNode.....	90
Appendix C	Virtual Transport Layer Implementation Details	93
C.1	VirtualUDP Class: The Technical Blueprint.....	93
C.2	Implementation Details of VirtualUDP Core Functions	96
C.2.1	Constructor Method (__init__)	96
C.2.2	Binding Method (bind)	97
C.2.3	Sending Method (sendto).....	97
C.2.4	Receiving and Demultiplexing Method (_receive_from_network)	99
C.2.5	Data Retrieval Method (recvfrom).....	100

Appendix D Virtual Application Layer Implementation Details

102

D.1	Architectural Blueprint of the Application Layer	102
D.2	Implementation Details of Application Tasks	104
D.2.1	Initialization and Orchestration.....	104
D.2.2	Server Logic (run_server_logic)	105
D.2.3	Client Logic (run_client_tasks).....	106

Appendix A Optimized Link State Routing (OLSR)

Protocol

Therefore, this appendix establishes the foundational theoretical concepts of OLSR. The focus is placed solely on the core and auxiliary functions that were realized in our software, as these details provide the necessary theoretical context for understanding the framework's design and architecture, which is the subject of Chapter 3

OLSR is a mature and widely recognized proactive routing protocol for Mobile Ad-Hoc Networks (MANETs), representing a significant optimization of classical link-state algorithms for the challenges of wireless, dynamic environments. Its complexity, stateful nature, and well-defined mechanisms for neighbor sensing, topology dissemination, and route computation make it an ideal candidate to demonstrate that an entire network-layer protocol can be successfully lifted into the application layer.

A.1 RFC series status and where RFC 3626 sits

The Request For Comments (RFC) series categorizes documents by their status, such as "Standards Track," "Informational," or "Experimental." It is significant to note that RFC 3626, which defines OLSR, is explicitly published with an Experimental status.

This classification is fundamentally important to this research methodology. Unlike a "Standards Track" protocol where strict adherence is paramount, an "Experimental" RFC inherently permits and encourages modifications, extensions, and alternative implementation

strategies for research purposes. It signifies that the protocol, while well-defined, is not considered a finalized standard and is open to further investigation.

A.2 Core Principles of OLSR

OLSR is a proactive, table-driven routing protocol developed specifically for Mobile Ad-Hoc Networks (MANETs). Unlike classic link state protocols, OLSR introduces significant optimizations to minimize control traffic overhead, making it well-suited for large and dense mobile networks.

1- Multipoint Relays (MPR)

The central concept in OLSR is the use of Multipoint Relays (MPRs) as shown in Figur A.1. A node selects a subset of its symmetric one-hop neighbors as MPRs. The primary functions of MPRs are twofold:

- **Optimized Flooding:** Only nodes that have been selected as MPRs are responsible for retransmitting (forwarding) control messages throughout the network. This optimization substantially reduces the number of redundant broadcast transmissions compared to classical flooding.
- **Partial Link State Diffusion:** MPRs are solely responsible for declaring link state information in the network. Specifically, a node selected as an MPR **MUST** declare links to the nodes that selected it, known as its MPR Selectors. This partial link state information is sufficient for all nodes to compute shortest path routes (in terms of hop count).

2- The Node and Main Address

Every node participating in the OLSR domain is uniquely identified by its Main Address, which is an IP address used as the originator address in all control traffic emitted by that node. For nodes with a single OLSR interface, this address must be the interface IP address; for multi-interface nodes, one interface IP address is chosen as the main address.

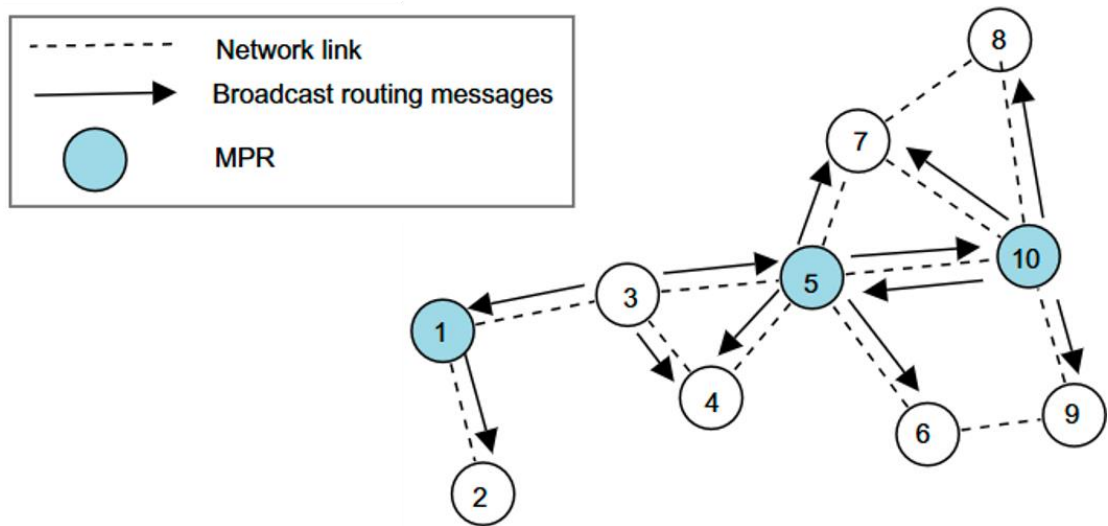


Figure A. 1: OLSR routing protocol showing selection of MPRs.

A.3 OLSR Packet Structure and Integrity

OLSR utilizes a unified packet format for all control traffic, allowing multiple messages of different types (e.g., HELLO, TC) to be encapsulated within a single UDP datagram (known as "piggybacking"). Figure A.2 Illustrated in this packet format, while Table A.1 provides a detailed explanation of the individual fields within the packet. OLSR packet consist of three things:

- **Packet Header:** Contains general information about the entire packet, most notably the Packet Length and the Packet Sequence Number (PSN), which is incremented with every new packet transmitted by the node.
- **Message Header:** Precedes each encapsulated message. Key fields include the Message Type (e.g., 1 for HELLO, 2 for TC), Validity Time (Vtime) indicating how long the information is valid, Originator Address (the node's main address, which must never change during retransmission), Time To Live (TTL) limiting the maximum number of hops, Hop Count tracking the traversed distance, and Message Sequence Number (MSN) used by the originator to identify message freshness.
- **Message Body:** Contains protocol-specific data, whose structure varies by message type (e.g., neighbor lists in HELLO, advertised links in TC). OLSR ensures integrity and freshness through sequence number checks: messages with an older or duplicate sequence are discarded, and those with $TTL \leq 1$ are not retransmitted.

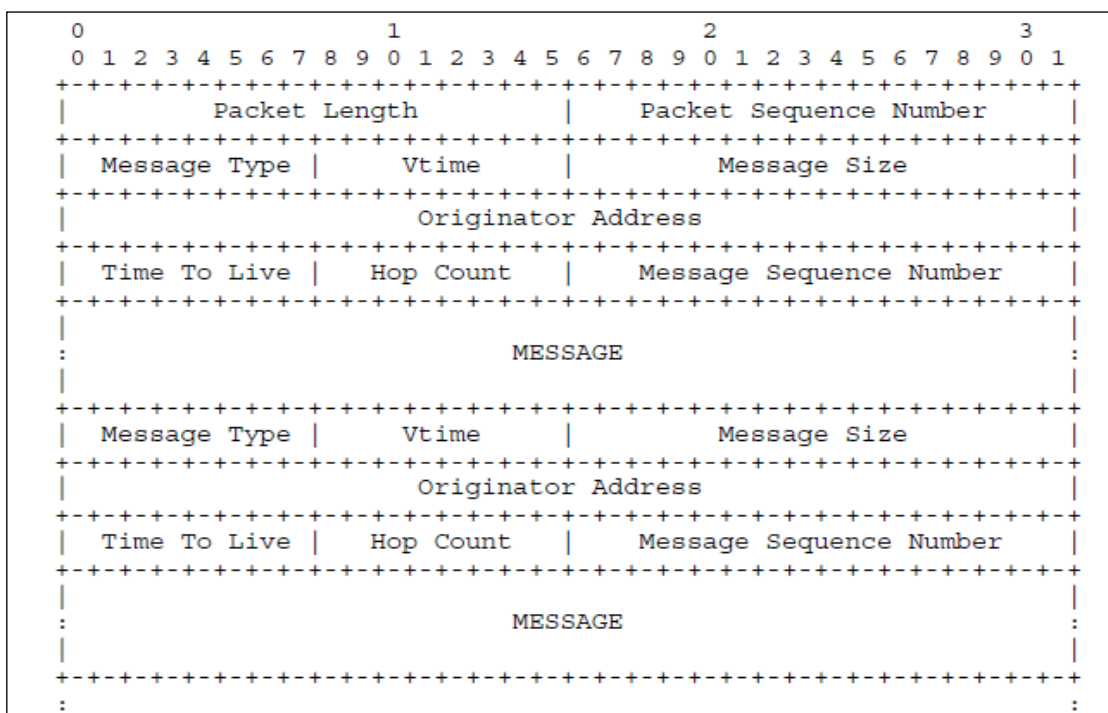


Figure A. 2: The unified OLSR packet structure

Table A. 1: The unified OLSR packet fields (RFC 3626)

Component	Field	Meaning & Units
Packet Header	Packet Length	Total bytes of the OLSR packet.
	Packet Sequence Number (PSN)	Per-packet identifier (node-local).
Message Header	Message Type	Message types: 1=HELLO, 2=TC, 3=MID, 4=HNA.
	Vtime	Encoded validity time (seconds) indicating how long the information is valid.
	Message Size	Bytes from Message Type to the next message (or packet end).
	Originator Address	Main address (IP) of the node that created the message.
	TTL	Max hops remaining.
	Hop Count	Hops already traversed.
	Message Sequence Number (MSN)	Per-originator sequence number.
Message Payload	Message	Message body, determined by type field in message header

A.4 Core and Auxiliary Control Messages

The protocol relies on four primary message types (HELLO, TC, HNA, and MID). Each type serves a distinct role in neighbor discovery, topology dissemination, gateway advertisement, and interface association. In the developed system, these theoretical roles were mirrored by implementing both the reception and emission of each message type, ensuring that the control plane behavior described in the standard was reproduced at the application layer. The following points provide a brief explanation of each message type:

1. HELLO Messages (Core):

HELLO messages are a core component of the OLSR protocol, used exclusively for link sensing and neighbor detection. To ensure they are only processed by immediate neighbors, these messages are always broadcast locally with a Time-to-Live (TTL) value of 1. Their primary function is to populate the node's Neighbor Set and 2-hop Neighbor Set. This is achieved by advertising the originator's own link list and its selected Multi-Point Relays (MPRs). Furthermore, HELLO messages are instrumental in signaling MPR selection, which in turn allows receiving nodes to populate their MPR Selector Set. The specific format of the HELLO message body is illustrated in Figure A.3, with a detailed explanation of its fields provided in Table A.2 bellow.

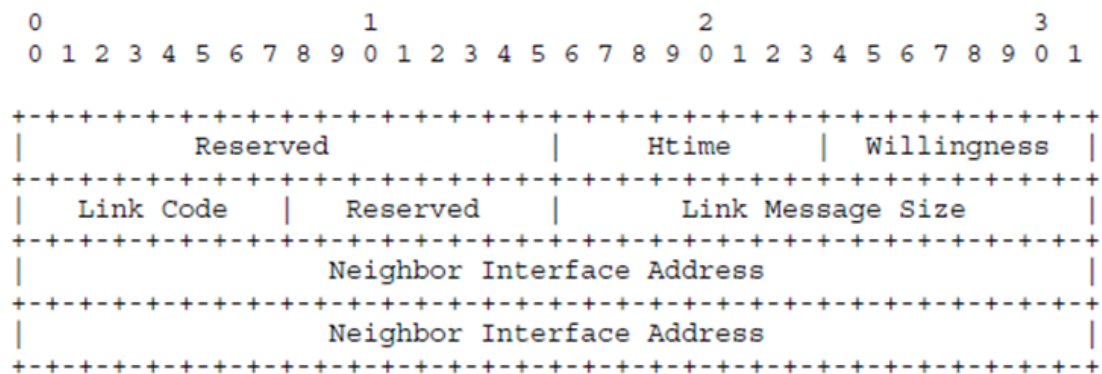


Figure A.3: HELLO message body format

Table A. 2: Description of HELLO message body fields

Field	Meaning / Units
Reserved	This 16-bit field must be set to zero to maintain compliance with the protocol specification
Htime	Specifies the HELLO emission interval in seconds.
Willingness	Node's willingness to forward on behalf of others.
Link Code	An 8-bit field that, encodes both the 2-bit Neighbor Type (defining the neighbor's status relative to the sender, i.e., NOT_NEIGH, SYM_NEIGH, MPR_NEIGH) and the 2-bit Link Type (defining the link status, e.g., ASYM_LINK and SYM_LINK)
Link Message Size	The size in bytes of the current link information block. Measured from this Link Code to the next Link Code (or end).
Neighbor Interface Address	The IP address of a neighbor node's interface. This address is listed to declare the neighbor set and associated link status to the receiving node.

2. Topology Control (TC) Messages (Core):

Beyond local neighbor discovery, OLSR utilizes Topology Control (TC) messages for disseminating link-state information throughout the network. Unlike the locally-scoped HELLO messages, TC messages are designed for network-wide topology discovery. They are generated exclusively by nodes that have been selected as Multi-Point Relays (MPRs) and serve to advertise the links between the MPR and its MPR selectors. This crucial information is then used to populate the Topology Information Base on each node. To ensure complete reachability, TC messages are flooded across the entire network, typically with a Time-to-Live (TTL) of 255, and are forwarded efficiently by other MPRs. The structure of the TC message body is

shown in Figure A.4. Table A.3 details the fields of this message and those of the subsequent messages.

3. Multiple Interface Declaration (MID) Messages (Auxiliary):

In addition to the core message types, OLSR defines auxiliary messages to handle specific network configurations. The Multiple Interface Declaration (MID) message, for instance, addresses scenarios involving nodes with multiple network interfaces. Its purpose is to allow a multi-homed node to declare its secondary interface addresses and associate them with its main address. This declaration populates the Multiple Interface Association Information Base on other nodes, ensuring that all of a node's various addresses are correctly mapped to a single identity for routing calculations. The format of the MID message is illustrated in Figure A.5 and Table A.3 details the fields of this message and those of the subsequent message.

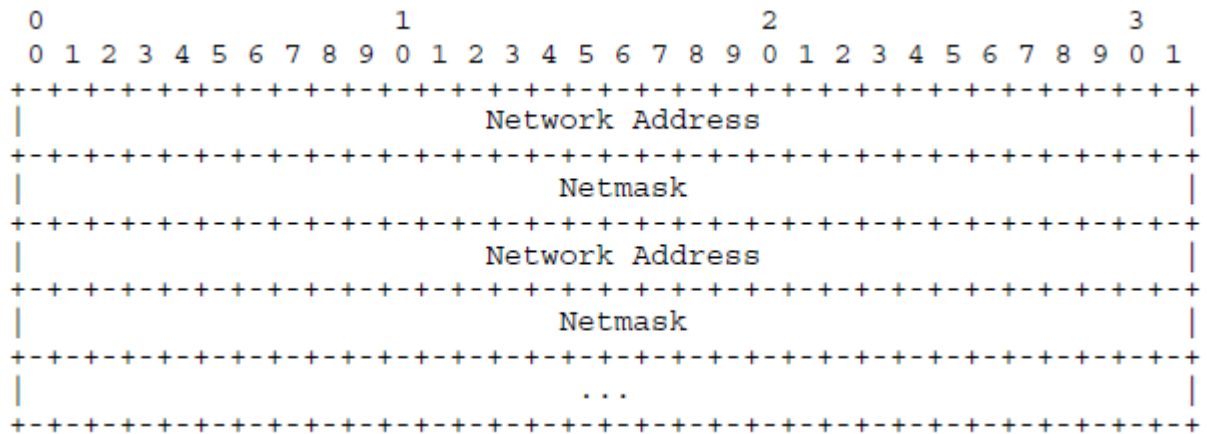


Figure A. 5:HNA Message body fields

4. Host and Network Association (HNA) Messages (Auxiliary):

Furthermore, the Host and Network Association (HNA) message provides a mechanism for an OLSR node to act as a gateway to external, non-OLSR networks. This auxiliary message is used to inject external routing information into the OLSR domain. It functions by advertising network address and netmask pairs, which are then stored in the Host and Network Association Information Base of nodes within the network, effectively enabling connectivity to external domains.

Table A. 3: TC, MID and HNA message bodies

Message	Field	Meaning / Units
TC	ANSN	The Advertised Neighbor Sequence Number is a sequence identifier used to indicate the freshness of the topology links declared in the TC message
	Advertised Neighbor Main Address	Main addresses of neighbors in the advertised link set.
MID	OLSR Interface Address	Interface addresses (excluding main address) of a multi-interface node.
HNA	(Network Address, Netmask) pairs	External reachability announced by gateways.

A.5 Protocol Constants and Timers

The operational stability and efficiency of OLSR are intrinsically linked to the correct configuration of its periodic transmission intervals and the associated holding times. These constants govern the protocol's responsiveness to topology changes and its overhead footprint. Table A.4 summarizes the mandated or recommended default values for the constants used in RFC 3626 specification.

Table A. 4:OLSR timing and validity parameters

Parameter	Default Value	Definition / Relation
HELLO_INTERVAL	2 seconds	Period between HELLO message transmissions.
REFRESH_INTERVAL	2 seconds	The predetermined refreshing period within which each link/neighbor MUST be cited at least once in HELLO messages. Equal to HELLO_INTERVAL by proposed value.
NEIGHB_HOLD_TIME	$3 \times \text{HELLO_INTERVAL}$	The validity duration for neighbor entries and link information.
TC_INTERVAL	5 seconds	Period for Topology Control (TC) message emission by MPRs.
TOP_HOLD_TIME	$3 \times \text{TC_INTERVAL}$	The validity duration for topology tuples in the Topology Set.
MID_INTERVAL	5 seconds	Period for Multiple Interface Declaration (MID) message emission (if the node has multiple OLSR interfaces).
MID_HOLD_TIME	$3 \times \text{MID_INTERVAL}$	The validity duration for entries in the Interface Association Information Base (MID tuples).
HNA_INTERVAL	5 seconds	Period for Host and Network Association (HNA) message

		emission (if the node serves as a gateway to external networks).
HNA_HOLD_TIME	$3 \times \text{HNA_INTERVAL}$	The validity duration for entries in the Host and Network Association Information Base (HNA tuples).

A.6 Routing Table Construction

The final output of the OLSR process is the Routing Table, which provides the optimal (shortest hop count) path to every known destination in the network. The routing table is computed based on the information stored in the local link set (symmetric neighbors) and the Topology Information Base (MPR selector links). The calculation involves running a shortest path algorithm (such as Dijkstra's algorithm) on a directed graph constructed from two types of arcs:

1. Arcs from node X to its symmetric one-hop neighbors Y (X -> Y).
2. Arcs from an MPR U to its MPR selector V (U -> V), derived from the Topology Set (T_last_addr -> T_dest_addr).

Routes to external networks, advertised via HNA messages, are integrated into the routing table by extending the shortest path found to the associated gateway node (HNA originator) by one additional hop. The routing table must be recalculated whenever any of the contributing information bases change.

A.7 The Distinction Between Control and Data Planes

A crucial aspect of the OLSR specification, as defined in RFC 3626, is its exclusive focus on the routing control plane. The standard meticulously defines the mechanisms by which nodes discover their

local topology, disseminate this information, and ultimately compute and maintain their routing tables. It provides explicit rules for how these tables are populated and updated, for instance, when processing HNA messages or reacting to changes in network topology.

However, it is important to note that RFC 3626 does not standardize the data forwarding mechanism itself. The protocol's responsibility ends once the routes are computed and handed over to the underlying IP stack of the operating system. The actual process of forwarding user data packets is considered an implementation detail left to the IP stack, which utilizes the routes provided by OLSR. This distinction is evident in the normative sections of the RFC, which are primarily concerned with (a) the structure and fields of the routing table, and (b) the procedures for adding or modifying its entries based on incoming control traffic like HELLO and TC messages.

Appendix B Virtual Network Layer Implementation

Details

This appendix provides a comprehensive technical reference for the user-space OLSR framework, complementing the high-level design and architectural discussions presented in Chapter 3. While the main chapter focuses on the "why" behind the design decisions, this appendix details the "how" of the implementation.

The following sections contain the granular specifications, data structures, and procedural logic that constitute the core of the `OlsrNode` class and its associated components. This includes detailed tables of class attributes and methods, the precise structure of OLSR packets and

messages, and the step-by-step logic for core processes such as packet assembly, integrity validation, and message processing. This content is intended to serve as a definitive blueprint, providing the necessary level of detail for a thorough understanding of the framework's source code and operational mechanics.

B.1 OlsrNode Class: The Complete Technical Blueprint

This section provides the complete technical blueprint for the `OlsrNode` class, which was introduced conceptually in Section The following components serve as the definitive implementation reference, detailing the class's structure, its data storage mechanisms (attributes), and its core operational logic (methods).

The Unified Modeling Language (UML) class diagram presented in Figure B.1 offers a visual overview of the class architecture, illustrating the relationships between its primary attributes and methods. Following the diagram, Table B.1 provides an exhaustive list of the class attributes, detailing their data types and functional descriptions. Finally, Table B.2 documents the key methods, outlining their specific roles and responsibilities within the OLSR protocol's lifecycle.

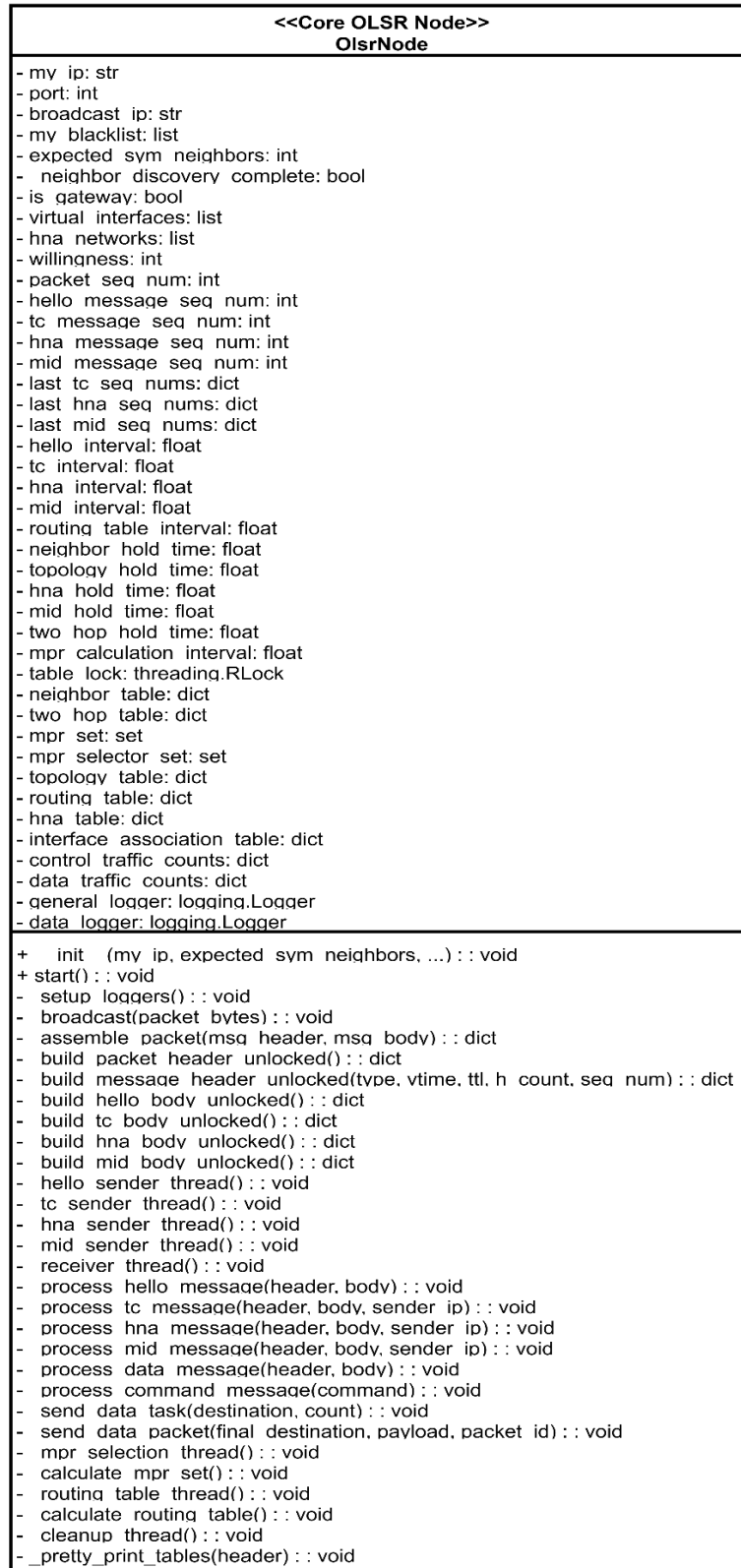


Figure B. 1: Architectural Overview of the OlsrNode Class

Table B. 1 Key Attributes of the OlsrNode Class

Category	Attribute Name	Type	Description
Configuration & ID	my_ip	Str	The node's own IP address, serving as the node's "main address".
	port	Int	The UDP port number used for OLSR communications (default 698).
	broadcast_ip	Str	The broadcast IP address used for sending OLSR packets.
	my_blacklist	List	List of IP addresses of nodes whose messages should be ignored, used to implement isolation rules (forming desired topology).
	expected_sym_neighbors	Int	The number of symmetric neighbors expected to signal completion of neighbor discovery.
	_neighbor_discovery_complete	Bool	Flag indicating whether initial neighbor discovery has been completed.
	is_gateway	Bool	Determines if the node acts as a gateway to external non-OLSR networks.

Timers & Hold Times	<code>virtual_interfaces</code>	List	List of IP addresses for the node's additional interfaces, used in MID messages.
	<code>hna_networks</code>	List	List of external networks (in network/netmask format) advertised if the node acts as a gateway.
	<code>willingness</code>	Int	The node's willingness to forward traffic for other nodes (ranging from 0 to 7).
	<code>hello_interval</code>	Float	Time interval between HELLO message transmissions.
	<code>tc_interval</code>	Float	Time interval between TC message transmissions.
	<code>hna_interval</code>	Float	Time interval between HNA message transmissions.
	<code>mid_interval</code>	Float	Time interval between MID message transmissions.
	<code>routing_table_interval</code>	Float	Time interval between routing table recalculations.
	<code>neighbor_hold_time</code>	Float	Duration for which neighbor information is considered valid.

	topology_hold_time	Float	Duration for which topology information is considered valid.
	hna_hold_time	Float	Duration for which HNA information is considered valid.
	mid_hold_time	Float	Duration for which MID information is considered valid.
	two_hop_hold_time	Float	Duration for which two-hop neighbor information is considered valid.
	mpr_calculation_interval	Float	Time interval between MPR set recalculations.
Sequence Numbers	packet_seq_num	Int	Global packet sequence number, incremented for each OLSR packet sent.
	hello_message_seq_num	Int	Sequence number for HELLO messages originated by the node.
	tc_message_seq_num	Int	Sequence number for TC messages originated by the node.
	hna_message_seq_num	Int	Sequence number for HNA messages originated by the node.

	mid_message_seq_num	Int	Sequence number for MID messages originated by the node.
	last_tc_seq_nums	Dict	Last TC sequence number received from each originator.
	last_hna_seq_nums	Dict	Last HNA sequence number received from each originator.
	last_mid_seq_nums	Dict	Last MID sequence number received from each originator.
Data Structures (Information Bases)	table_lock	threading. RLock	Reentrant lock ensuring thread-safe access to shared tables.
	neighbor_table	Dict	Stores information about one-hop neighbors.
	two_hop_table	Dict	Stores information about two-hop neighbors.
	mpr_set	Set	Set of IP addresses of neighbors selected as Multipoint Relays (MPRs).
	mpr_selector_set	Set	Set of IP addresses of neighbors that selected this node as their MPR.

	topology_table	Dict	Stores discovered network topology information.
	routing_table	Dict	Routing table containing best paths to all known destinations.
	hna_table	Dict	Stores information about external networks reachable via HNA gateways.
	interface_association_table	Dict	Stores multi-interface association information for nodes.
Metrics & Loggers	control_traffic_counts	Dict	Counters for control message traffic (HELLO, TC, HNA, MID).
	data_traffic_counts	Dict	Counters for data message traffic (DATA_SENT, DATA_FORWARDED, DATA_RECEIVED).
	general_logger	logging. Logger	Logger for general events and diagnostic messages.
	data_logger	logging. Logger	A dedicated logger for data traffic metrics for correctness and performance analysis (e.g., packet delivery, delay).

Table B. 2 Summary of Key OlsrNode Class Methods and Their Functions

Category	Method	Description
Initialization & Lifecycle Methods	<code>__init__(self, my_ip, expected_sym_neighbors, ...)</code>	The class constructor, which initializes all attributes upon OlsrNode object creation.
	<code>start(self)</code>	Initiates all necessary daemon threads for the node's operation and enters a main waiting loop.
	<code>_setup_loggers(self)</code>	Configures the general and data traffic event loggers.
Headers Builder Methods	<code>_assemble_packet(self, msg_header, msg_body)</code>	Assembles a message header and body into a complete OLSR packet structure.
	<code>_build_packet_header_unlocked(self)</code>	Constructs an OLSR packet header.
	<code>_build_message_header_unlocked(self, msg_type, vtime, ttl, hop_count, seq_num)</code>	Constructs an OLSR message header.

Message Body Builder Methods	<code>_build_hello_body_unlocked(self)</code>	Constructs the body of a HELLO message.
	<code>_build_tc_body_unlocked(self)</code>	Constructs the body of a TC message.
	<code>_build_hna_body_unlocked(self)</code>	Constructs the body of an HNA message.
	<code>_build_mid_body_unlocked(self)</code>	Constructs the body of a MID message.
Periodic Sender Threads	<code>_broadcast(self, packet_bytes)</code>	Used to send packets over the network via a UDP socket.
	<code>_hello_sender_thread(self)</code>	A thread that periodically sends HELLO messages for neighbor discovery.
	<code>_tc_sender_thread(self)</code>	A thread that periodically sends TC messages for topology dissemination (if the node is an MPR).
	<code>_hna_sender_thread(self)</code>	A thread that periodically sends HNA

		messages (if the node is a gateway).
	<code>_mid_sender_thread(self)</code>	A thread that periodically sends MID messages (if the node has multiple interfaces).
	<code>_receiver_thread(self)</code>	The thread that continuously receives incoming packets, decodes them, and dispatches them to appropriate processing functions.
Receiver Thread & Message Processing Methods	<code>_process_hello_message(self, header, body)</code>	Processes received HELLO messages to update neighbor and MPR selector tables.
	<code>_process_tc_message(self, header, body, sender_ip)</code>	Processes received TC messages to update the topology table and forwards them if necessary.

<code>_process_hna_message(self, header, body, sender_ip)</code>	Processes received HNA messages to update the HNA table and forwards them.
<code>_process_mid_message(self, header, body, sender_ip)</code>	Processes received MID messages to update the interface association table and forwards them.
<code>_process_data_message(self, header, body)</code>	Processes incoming DATA messages, either receiving them at the destination or forwarding them.
<code>_process_command_message(self, command)</code>	Processes special command messages (e.g., to initiate PDR tests).
<code>_send_data_task(self, destination, count)</code>	A task to send a specified number of data packets to a given destination.
<code>_send_data_packet(self, final_destination, payload, packet_id)</code>	Construct and sends a single data packet to a final destination.

Table Maintenance & Calculation Threads/Functions	<code>_mpr_selection_thread(self)</code>	A thread that periodically calculates the MPR set.
	<code>_calculate_mpr_set(self)</code>	A function that computes the MPR set based on neighbor and two-hop neighbor tables.
	<code>_routing_table_thread(self)</code>	A thread that periodically calculates the routing table.
	<code>_calculate_routing_table(self)</code>	A function that computes the routing table using Dijkstra's algorithm.
	<code>_cleanup_thread(self)</code>	A thread that periodically removes expired entries from various tables.
Utility / Debugging Functions	<code>_pretty_print_tables(self, header)</code>	Used to print the contents of various tables in a formatted manner for debugging and monitoring purposes.

B.2 Detailed Packet and Message Structures

This section provides the detailed specifications for the construction of OLSR packets and their constituent messages within the framework. It documents both the final dictionary structure of each component and the specific generator methods from the `OlsrNode` class responsible for their creation. As outlined in Chapter 3, this implementation uses Python dictionaries for internal representation and JSON for network transmission, a decision that prioritizes flexibility and readability.

The framework adheres to the hierarchical structure defined in RFC 3626, where a single packet contains a packet header and a list of one or more messages. Each message, in turn, is composed of a message header and a message body. The overall conceptual structure of an OLSR packet in this implementation is as follows:

OLSR Packet (Python Dictionary)

```
"packet_header": { ... Packet Header Fields ... }
"messages": [
    {
        "message_header": { ... Message Header Fields ... }
        "message_body": { ... Message Body Fields ... }
    }
    // Potentially more messages in the list
]
```

B.2.1 Message Body Structures

The Message Body dictionary (`message_body`) holds the specific control information pertinent to the message type. This content varies significantly based on whether the message is a HELLO, TC, HNA, MID, or a custom DATA packet. These dictionaries are typically created by dedicated helper functions within the `OlsrNode` class. The following points provide a

detailed specification for each message body type. For each type, the documentation is structured to present three key pieces of information:

- **Purpose:** A brief description of the message's role in the OLSR protocol.
- **Dictionary Structure:** The definitive key-value structure of the Python dictionary used to implement the message. This structure serves as the practical, code-level counterpart to the conceptual binary format fields described in Chapter 2). It represents the blueprint that is later serialized into a JSON string for transmission.
- **Generator Method:** The specific Python code snippet from the `OlsrNode` class responsible for constructing the dictionary.

1- HELLO Message Body

- **Purpose:** Used for link sensing, neighbor detection, and MPR signaling.
- **Dictionary Structure:**

```
{
    "htime":          # HELLO emission interval
    "willingness":    # Node's willingness to forward traffic
    "links": [
        {
            "link_code": "HEARD",      # Type of link with a neighbor (e.g.,
HEARD, SYM, ASYM)
            "neighbor_ip":             # IP address of the neighbor
        },
        // ... more link entries
    ],
    "mpr_set": list      # List of IPs of nodes selected as MPRs by this
node
}
```

Example: A HELLO message body might appear as:

```
{
    "htime": 2.0,
    "willingness": 3,
    "links": [
        {"link_code": "HEARD", "neighbor_ip": "192.168.10.2"},
        {"link_code": "HEARD", "neighbor_ip": "192.168.10.3"}
    ]
}
```

```
],
  "mpr_set": ["192.168.10.2"]
}
```

- **Generator Method (`_build_hello_body_unlocked`):** This method constructs the HELLO message body by gathering current node state. It retrieves the list of one-hop neighbors from `self.neighbor_table` and the current Multipoint Relay set from `self.mpr_set`.

```
def _build_hello_body_unlocked(self):
    return {
        "htime": self.hello_interval,
        "willingness": self.willingness,
        "links": [{"link_code": "HEARD", "neighbor_ip": ip} for ip in
self.neighbor_table.keys()],
        "mpr_set": list(self.mpr_set)
    }
```

2- Topology Control (TC) Message Body

- **Purpose:** Used to disseminate topology information (MPR selector links).
- **Dictionary Structure:**

```
{
  "mpr_selectors": list    # List of IPs of nodes that have selected this
node as an MPR
}
```

Example:

```
{
  "mpr_selectors": ["192.168.10.3"]
}
```

- **Generator Method (`_build_tc_body_unlocked`):** This method constructs the TC message body by advertising the set of nodes that

have selected the current node as an MPR, which is stored in `self.mpr_selector_set`.

```
def _build_tc_body_unlocked(self):  
    return {"mpr_selectors": list(self.mpr_selector_set)}
```

3- Host and Network Association (HNA) Message Body

- Purpose: Allows gateway nodes to advertise reachability to external networks.
- Dictionary Structure:

```
{  
    "networks": list          # List of networks (e.g., "0.0.0.0/0") that this  
                             # node gateways to  
}
```

Example:

```
{  
    "networks": ["0.0.0.0/0"]  
}
```

- Generator Method (`_build_hna_body_unlocked`): This method constructs the HNA message body by retrieving the list of external networks from the `self.hna_networks` attribute.

```
def _build_hna_body_unlocked(self):  
    return {"networks": self.hna_networks}
```

4- Multiple Interface Declaration (MID) Message Body

- Purpose: Declares multiple interfaces associated with a single node's main address.
- Dictionary Structure:

```
{  
    "virtual_interfaces": list          # List of virtual interface IPs  
associated with this node's main address  
}
```

Example:

```
{  
    "virtual_interfaces": ["10.10.10.2", "10.20.20.2"]  
}
```

- Generator Method (`_build_mid_body_unlocked`): This method constructs the MID message body by retrieving the list of secondary interface addresses from the `self.virtual_interfaces` attribute.

```
def _build_mid_body_unlocked(self):  
    return {"virtual_interfaces": self.virtual_interfaces}
```

5- Data Message Body (Custom Extension)

- Purpose: Carries application-layer payloads and path-tracking information for data packets.
- Dictionary Structure:

```
{  
    "final_destination": str,          # The ultimate IP address of the data  
packet's recipient  
    "payload": str,                   # The actual data content  
    "packet_id": int,                 # A unique identifier for the data packet  
}
```

```

    "creation_timestamp": float, # Timestamp when the packet was created
    "path_taken": list           # List of IPs of nodes the packet has
                                # traversed
}

```

Example:

```

{
    "final_destination": "192.168.10.5",
    "payload": "Test packet 1",
    "packet_id": 1,
    "creation_timestamp": 1678886400.12345,
    "path_taken": ["192.168.10.1"]
}

```

- **Generator Context (`_send_data_packet`):** Unlike control messages, the DATA message body is constructed dynamically within the `_send_data_packet` function just before transmission, using parameters passed to the function and the node's current state.

```

# Snippet from _send_data_packet function
data_body = {
    "final_destination": final_destination,
    "payload": payload,
    "packet_id": packet_id,
    "creation_timestamp": time.time(),
    "path_taken": [self.my_ip]
}

```

B.2.2 Construction of the Message Header

The `message_header` dictionary contains common control information applicable to any encapsulated message. It is generated by the `_build_message_header_unlocked()` method.

- **Purpose:** To provide essential metadata for message routing, validity, and identification, as defined by RFC 3626.
- **Dictionary Structure:**

```
{
    "msg_type": str,      # Type of the message (e.g., "HELLO", "TC", "HNA",
                          "MID", "DATA")
    "vtime": float,       # Validity time for the information in the
                          message (e.g., neighbor_hold_time)
    "msg_size": int,      # Size of the message (calculated
                          dynamically, see Section 3.5)
    "originator_address": str, # Main IP address of the node that
                          originally generated this message
    "ttl": int,           # Time To Live; max hops before retransmission
                          ceases
    "hop_count": int,     # Number of hops the message has traversed
    "msg_seq_num": int    # Sequence number for the message, incremented by
                          originator
}
```

Example for a HELLO Message Header:

```
{
    "msg_type": "HELLO",
    "vtime": 6.0, // Corresponds to neighbor_hold_time (hello_interval * 3)
    "msg_size": 0, // Placeholder, updated dynamically
    "originator_address": "192.168.10.1",
    "ttl": 1, // HELLO messages are only for 1-hop neighbors, not
forwarded
    "hop_count": 0,
    "msg_seq_num": 10
}
```

- **Generator Method** (`_build_message_header_unlocked`): This method constructs the header using parameters passed by the calling sender thread. The `originator_address` is automatically set to the node's own IP, while `msg_size` is initialized as a placeholder to be calculated dynamically during packet assembly.

```
def _build_message_header_unlocked(self, msg_type, vtime, ttl, hop_count,
seq_num):
    return {
        "msg_type": msg_type,
        "vtime": vtime,
        "msg_size": 0, # Placeholder, calculated later
        "originator_address": self.my_ip,
```

```

        "ttl": ttl,
        "hop_count": hop_count,
        "msg_seq_num": seq_num
    }

```

B.2.3 Construction of the Packet Header

The `packet_header` dictionary contains metadata that applies to the entire OLSR packet, rather than a single message. It is generated by the `_build_packet_header_unlocked()` method.

- Purpose: To provide a unique sequence number for the entire packet and to specify its total length in bytes.
- Dictionary Structure:

```

{
    "packet_length": int,      # Total length of the packet in bytes
                               (calculated dynamically, see Section 3.5)
    "packet_seq_num": int      # Unique sequence number for the entire OLSR
                               packet, incremented for each new packet
}

```

Example:

```

{
    "packet_length": 0,        // Placeholder, updated dynamically
    "packet_seq_num": 25
}

```

- Generator Method (`_build_packet_header_unlocked`): This method constructs the header using the node's global `packet_seq_num`. The `packet_length` field is initialized as a placeholder, which is dynamically calculated after the full packet has been serialized to a JSON string.

```

def _build_packet_header_unlocked(self):
    return {
        "packet_length": 0, # Placeholder, calculated later
        "packet_seq_num": self.packet_seq_num
    }

```

```
}
```

B.3 Packet Assembly Process

This section provides the detailed technical specification for the packet assembly process, as introduced in Section 3.3.3. It details the implementation of the `_assemble_packet` method, which is responsible for combining the individual packet header, message header, and message body dictionaries into a final, unified packet structure ready for serialization. See Figure B.2.

- **Purpose:** To orchestrate the final assembly of an OLSR packet, including the generation of the packet header and the dynamic calculation of message and packet sizes.
- **Implementation (`_assemble_packet` method):** The method executes a series of steps to construct the final packet dictionary. It first increments the global `packet_seq_num` and builds the packet header. It then encapsulates the provided message header and body into a single message structure. Crucially, it performs a dynamic size calculation by serializing the structures to JSON to determine their byte length, updating the `msg_size` and `packet_length` fields before returning the final, complete packet.

```
def _assemble_packet(self, msg_header, msg_body):  
    # 1. Increment global packet sequence number  
    self.packet_seq_num += 1  
  
    # 2. Build the main packet header  
    packet_header = self._build_packet_header_unlocked()  
  
    # 3. Encapsulate the message header and body  
    full_message = {"message_header": msg_header, "message_body":  
msg_body}  
  
    # 4. Combine all parts into the final packet structure
```

```

    packet = {"packet_header": packet_header, "messages":
[full_message]}

    # 5. Perform dynamic size calculation and update placeholders
    message_str = json.dumps(packet)
    packet["packet_header"]["packet_length"] = len(message_str)
    packet["messages"][0]["message_header"]["msg_size"] =
len(json.dumps(msg_body))

    return packet

```

- **Example Output Structure:** The method returns a comprehensive Python dictionary. The example below illustrates the final structure for a packet containing a single HELLO message, with all size fields correctly populated.

```

{
  "packet_header": {
    "packet_length": 350, // Example calculated length
    "packet_seq_num": 25
  },
  "messages": [
    {
      "message_header": {
        "msg_type": "HELLO",
        "vtime": 6.0,
        "msg_size": 280, // Example calculated length of message
body
        "originator_address": "192.168.10.1",
        "ttl": 1,
        "hop_count": 0,
        "msg_seq_num": 10
      },
      "message_body": {
        "htime": 2.0,
        "willingness": 3,
        "links": [
          {"link_code": "HEARD", "neighbor_ip":
"192.168.10.2"},
          {"link_code": "HEARD", "neighbor_ip":
"192.168.10.3"}
        ],

```

```

    "mpr_set": ["192.168.10.2"]
  }
}
]
}

```

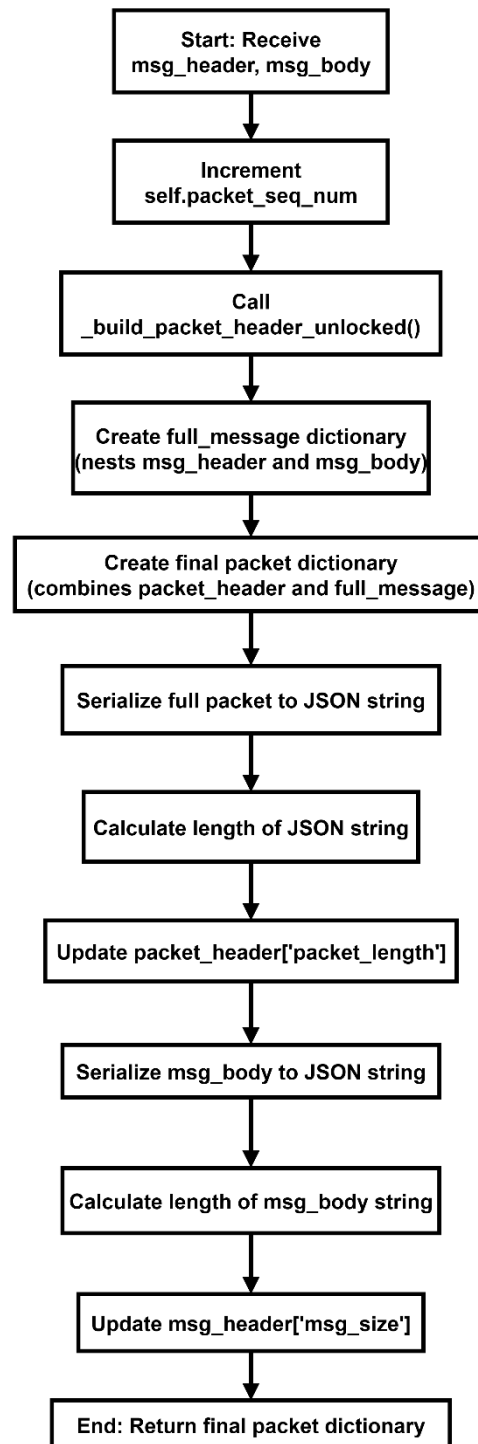


Figure B.2 Flowchart of the `_assemble_packet()` method

B.4 Packet Serialization and Transmission Specifications

This section provides the detailed implementation specifications for the packet serialization and transmission process, complementing the design overview in Section 3.4. It details the standardized procedure for converting a Python dictionary packet into a transmittable byte stream and the corresponding reception and deserialization process. See Figure B.3.

B.4.1 Serialization and Encoding: From Dictionary to Bytes

The core of the serialization process is a single, consistent line of Python code responsible for transforming the abstract packet dictionary into a byte stream. This is achieved through a two-step sequence:

- 1- `Serialization (json.dumps())`: The complete packet dictionary is converted into a JSON-formatted string.
- 2- `Encoding (.encode('utf-8'))`: The resulting JSON string is encoded into a sequence of bytes using the UTF-8 standard.

This sequence, `json.dumps(packet).encode('utf-8')`, is uniformly applied across the framework. The example below, taken from a typical sender thread, demonstrates its application in context.

```
# Snippet from a control message sender thread (e.g.,  
_hello_sender_thread)  
  
# 1. A complete packet dictionary is assembled  
packet = self._assemble_packet(header, body)  
  
# 2. The dictionary is serialized and encoded into a byte stream  
packet_bytes = json.dumps(packet).encode('utf-8')  
  
# 3. The byte stream is passed to the broadcast function for  
transmission  
self._broadcast(packet_bytes)
```

This same serialization logic is also employed by external components, such as the `test_runner.py` script, ensuring a unified data format for both internal control messages and external command messages.

B.4.2 Network Transmission: The `_broadcast` Method

The final transmission to the network is handled by the `_broadcast()` helper method. This function is responsible for configuring a UDP socket for broadcast communications and dispatching the byte stream to the network's broadcast address (`self.broadcast_ip`) on the designated OLSR port (698).

```
def _broadcast(self, packet_bytes):  
  
    # Transmits a byte stream over a broadcast UDP socket.  
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as  
sender_sock:  
  
        # Enable the broadcast option on the socket  
        sender_sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST,  
1)  
  
        # Send the byte stream to the broadcast address and OLSR port  
        sender_sock.sendto(packet_bytes, (self.broadcast_ip, self.port))
```

B.4.3 Reception and Deserialization: From Bytes to Dictionary

The reverse process occurs within the `_receiver_thread`. An incoming byte stream from the network socket is first decoded from UTF-8 and then deserialized (parsed) from a JSON string back into a Python dictionary, making it ready for processing by the framework's logic.

```
def _receiver_thread(self):  
    # ... (socket setup) ...  
    while True:  
        try:  
            # 1. Raw byte stream is received from the socket  
            # 2048 is the maximum size (in bytes) of the data packet
```

```
data, addr = receiver_sock.recvfrom(2048)

# 2. The byte stream is decoded and deserialized into a
dictionary
packet = json.loads(data.decode('utf-8'))

# ... (Packet processing logic follows) ...

except (json.JSONDecodeError, KeyError, TypeError):
    # Gracefully handles corrupted or non-JSON data
    Pass
```

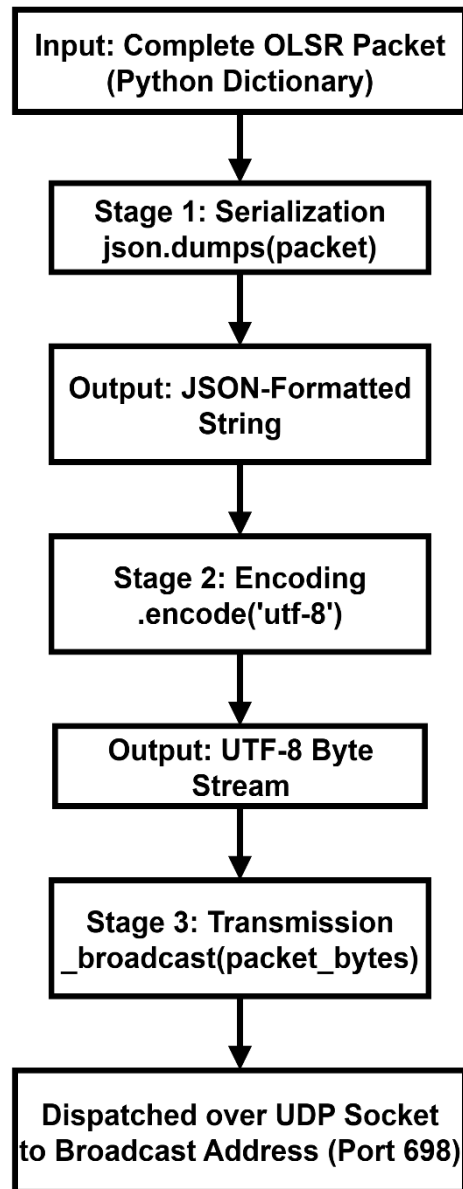


Figure B.3 The packet serialization and transmission process

B.5 Implementation of Packet Integrity Mechanisms

This section provides the detailed implementation specifications for the multi-layered integrity and validation checks performed by the framework, complementing the design overview presented in Section 3.3.5. The following subsections detail the implementation of both structural and protocol-specific validation mechanisms, the complete technical flow of which, encompassing packet reception and initial error handling, is visually illustrated in Figure B.4.

B.5.1 Structural and Syntactic Validation

This The first layer of defense is Structural and Syntactic Validation, which occurs immediately upon packet reception. This layer , ensures that only data conforming to the expected format is passed on for further processing. It is implemented through two sequential checks within the `_receiver_thread`:

1. The framework handles potential JSON decoding errors. Upon receiving a raw byte stream, it attempts to decode it from UTF-8 and then parse it into a Python dictionary using `json.loads()`. If the data is corrupted to the extent that it no longer forms a valid JSON structure (e.g., due to missing brackets or invalid characters), a `json.JSONDecodeError` is raised. The `_receiver_thread` explicitly catches this exception and silently discards the packet, preventing malformed data from causing an application crash.
2. Following a successful parse, the framework validates the packet's internal structure by handling Key and Type errors. It accesses dictionary fields using safe methods like `.get("key", default_value)`, which gracefully handles missing keys. The entire processing block is also protected against `KeyError` and `TypeError` exceptions, which might occur if a field contains data of an unexpected type (e.g., a string where an integer is expected). This provides a basic but effective form of schema validation, ensuring that structurally incomplete or type-inconsistent messages are also discarded rather than causing a runtime failure.

```
# Snippet from _receiver_thread
def _receiver_thread(self):
    # ... (socket setup) ...
    while True:
        try:
            data, addr = receiver_sock.recvfrom(2048)
            packet = json.loads(data.decode('utf-8'))

            # ... (further processing) ...
```

```
except (json.JSONDecodeError, KeyError, TypeError):
    # Silently discard corrupted or malformed packets
    pass
```

B.5.2 Logical and Protocol-Specific Validation

This second layer validates the message content against the rules and state of the OLSR protocol. The complete logic, from initial validation to message dispatch and forwarding checks, is depicted in Figure A.3.

1. Originator and Blacklist Validation

- **Implementation:** Before processing the messages within a packet, the `_receiver_thread` first checks for external **COMMAND** messages. For all standard OLSR messages, it validates that the originator is not the node itself and that the sender is not in the `self.my_blacklist`.

```
# Snippet from _receiver_thread
sender_ip = addr[0]

# ... (COMMAND check omitted for brevity) ...

# Blacklist validation (Framework Extension)
if sender_ip in self.my_blacklist:
    continue # Discard packet

for message in packet.get("messages", []):
    header = message.get("message_header", {})
    originator_ip = header.get("originator_address")

    # Originator validation
    if not originator_ip or originator_ip == self.my_ip:
        continue # Discard message

# ... (dispatch to message processors) ...
```

2. Duplicate Suppression (Control Messages)

- Implementation: Each message processing function for TC, HNA, and MID messages performs a sequence number check. It compares the incoming `msg_seq_num` with the last known sequence number for that originator (e.g., from `self.last_tc_seq_nums`). If the incoming number is not strictly greater, the function returns immediately, discarding the stale message.

```
# Snippet from _process_tc_message
def _process_tc_message(self, header, body, sender_ip):
    with self.table_lock:
        originator = header['originator_address']
        seq_num = header['msg_seq_num']

        if self.last_tc_seq_nums.get(originator, -1) >= seq_num:
            return # Discard stale or duplicate message

        self.last_tc_seq_nums[originator] = seq_num
        # ... (rest of processing) ...
```

3. Loop Detection (DATA Packets)

- Implementation: The `_process_data_message` function inspects the `path_taken` list in the body of custom DATA packets. If the node's own IP address is already present in the list, it identifies a loop and immediately discards the packet.

```
# Snippet from _process_data_message
def _process_data_message(self, header, body):
    with self.table_lock:
        path = body.get("path_taken", [])

        if self.my_ip in path:
            print(f"LOOP DETECTED for packet {body.get('packet_id')}. Dropping.")
            return # Discard looping packet

        path.append(self.my_ip)
        # ... (rest of processing) ...
```

4. Forwarding Logic Validation (TTL and MPR Check)

- **Implementation:** The processing functions for forwardable control messages (TC, HNA, MID) include a compound condition to check if `header['ttl'] > 1` and if the `sender_ip` is in the node's `self.mpr_selector_set`. This enforces OLSR's optimized flooding mechanism, as shown in the detailed processing flow of Figure B.5.

```
# Snippet from _process_tc_message
def _process_tc_message(self, header, body, sender_ip):
    with self.table_lock:
        # ... (message processing) ...

        if header['ttl'] > 1 and sender_ip in self.mpr_selector_set:
            header['ttl'] -= 1
            # ... (code to re-assemble and broadcast the packet) ...
```

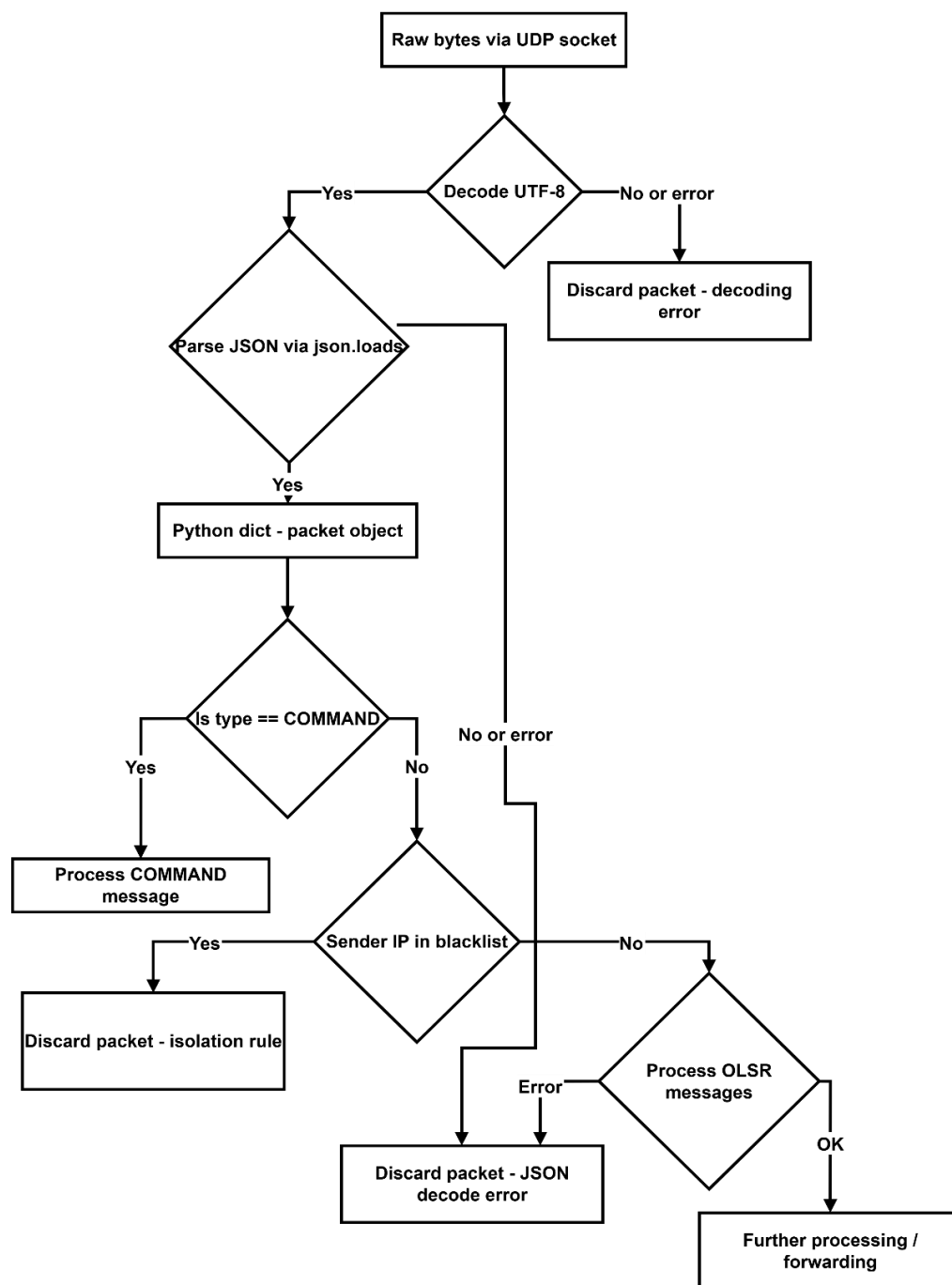


Figure B.4: Packet Reception Flow with Initial Error Handling

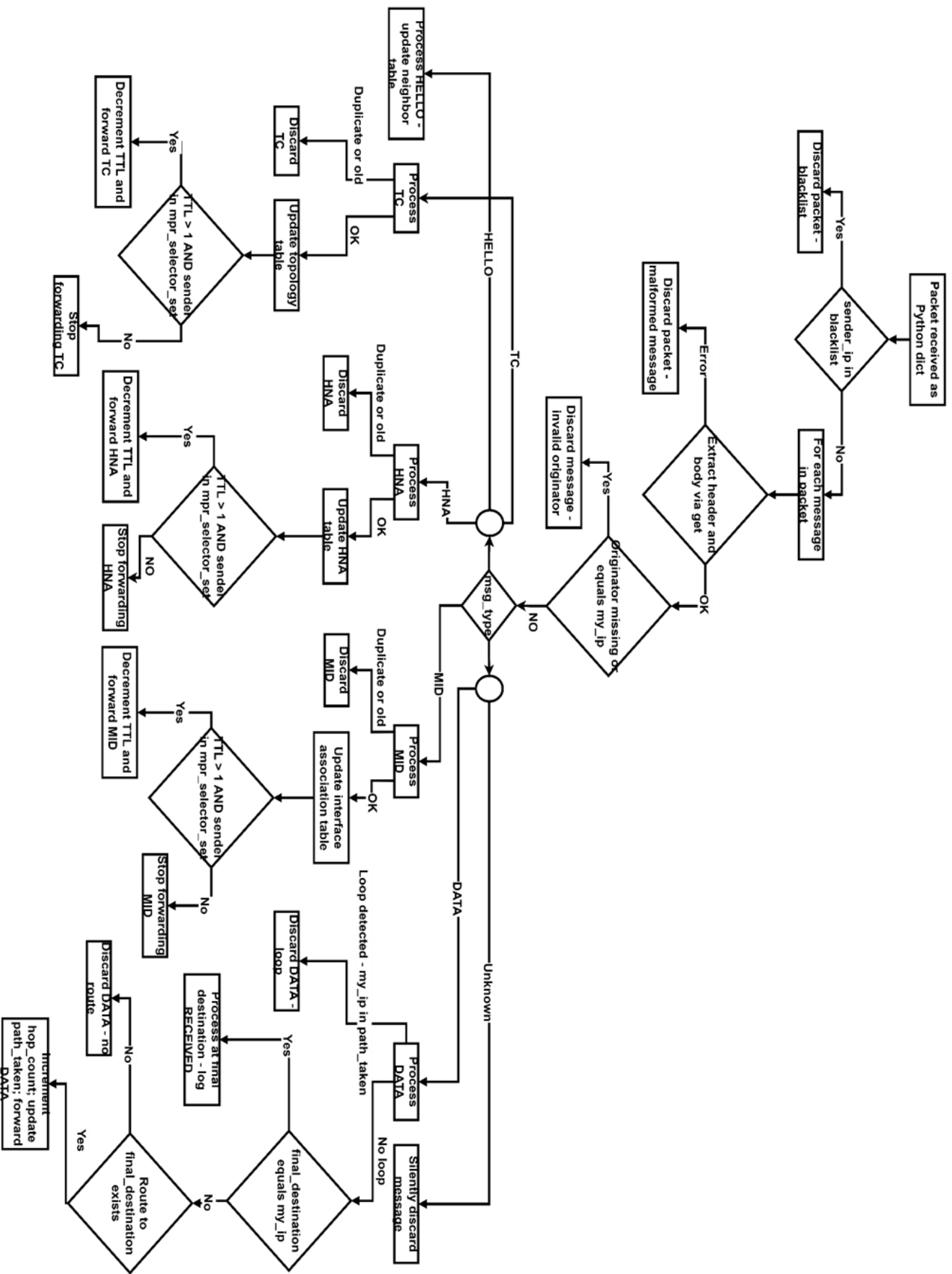


Figure B. 5: Detailed Message Processing and Integrity Checks.

B.6 Implementation of Message Transmission Logic

This section provides the detailed implementation of the thread-based sender architecture described in Section 3.3.6. Each core OLSR control message type is managed by a dedicated, long-running daemon thread that handles its periodic generation and broadcast.

B.6.1 HELLO Message Transmission (`_hello_sender_thread`)

This thread is responsible for the continuous, periodic broadcast of HELLO messages, which are essential for neighbor discovery on all nodes.

- **Purpose:** To discover and maintain one-hop neighbor relationships and signal MPR selection.
- **Implementation:** The thread operates in an infinite loop, sleeping for the duration of `self.hello_interval`. Upon waking, it acquires the `table_lock`, increments its sequence number and traffic counter, builds the HELLO message components, assembles the final packet, and broadcasts it. Figure B.6 illustrates the flowchart for HELLO Message Generation.

```
def _hello_sender_thread(self):
    while True:
        time.sleep(self.hello_interval)
        with self.table_lock:
            self.hello_message_seq_num += 1
            self.control_traffic_counts['HELLO'] += 1
            body = self._build_hello_body_unlocked()
            header = self._build_message_header_unlocked("HELLO",
self.neighbor_hold_time, 1, 0, self.hello_message_seq_num)
            packet = self._assemble_packet(header, body)

            self._broadcast(json.dumps(packet).encode('utf-8'))
            self.general_logger.info("HELLO_SENT")
```

B.6.2 Topology Control (TC) Message Transmission (`_tc_sender_thread`)

This thread handles the periodic broadcast of TC messages. Crucially, it includes logic to ensure that only nodes selected as MPRs will transmit these messages.

- **Purpose:** To disseminate link-state information about MPR selector sets, enabling all nodes to compute the network topology.
- **Implementation:** The thread sleeps for `self.tc_interval`. Its primary conditional check, if not `self.mpr_selector_set`: `continue`, ensures that TC message generation is skipped entirely unless the node has been selected as an MPR by at least one neighbor. If the condition is met, it proceeds with message construction and broadcast. Figure B.7 illustrates the flowchart for TC Message Generation.

```
def _tc_sender_thread(self):
    while True:
        time.sleep(self.tc_interval)
        with self.table_lock:
            # Conditional check: Only MPRs send TC messages
            if not self.mpr_selector_set:
                continue

            self.tc_message_seq_num += 1
            self.control_traffic_counts['TC'] += 1
            # ... (logic for updating local topology table omitted for
brevity) ...
            body = self._build_tc_body_unlocked()
            header = self._build_message_header_unlocked("TC",
self.topology_hold_time, 255, 0, self.tc_message_seq_num)
            packet = self._assemble_packet(header, body)

            self._broadcast(json.dumps(packet).encode('utf-8'))
            self.general_logger.info(f"TC_SENT, seq={self.tc_message_seq_num}")
    )
```

B.6.3 HNA and MID Message Transmission

`(_hna_sender_thread, _mid_sender_thread)`

These threads are responsible for auxiliary messages and are only activated if the node is configured with specific roles or attributes.

- `_hna_sender_thread`: This thread is started only if `self.is_gateway` is `True`. It periodically broadcasts HNA messages containing external network routes defined in `self.hna_networks`. Figure B.8 illustrates the flowchart for HNA Message Generation.
- `_mid_sender_thread`: This thread is started only if the `self.virtual_interfaces` list is not empty. It periodically broadcasts MID messages to declare the association between the node's main address and its other interface addresses. Figure B.9 illustrates the flowchart for MID Message Generation

The implementation of these threads follows the same pattern as the HELLO and TC senders, with their execution being conditional on the node's configuration at startup.

```
# Snippet from start() method showing conditional thread creation
# ...
threads = [
    # ... (HELLO, TC, Receiver threads) ...
]
if self.is_gateway:
    threads.append(threading.Thread(target=self._hna_sender_thread,
    daemon=True, name="HNASender"))
if self.virtual_interfaces:
    threads.append(threading.Thread(target=self._mid_sender_thread,
    daemon=True, name="MIDSender"))
# ...
```

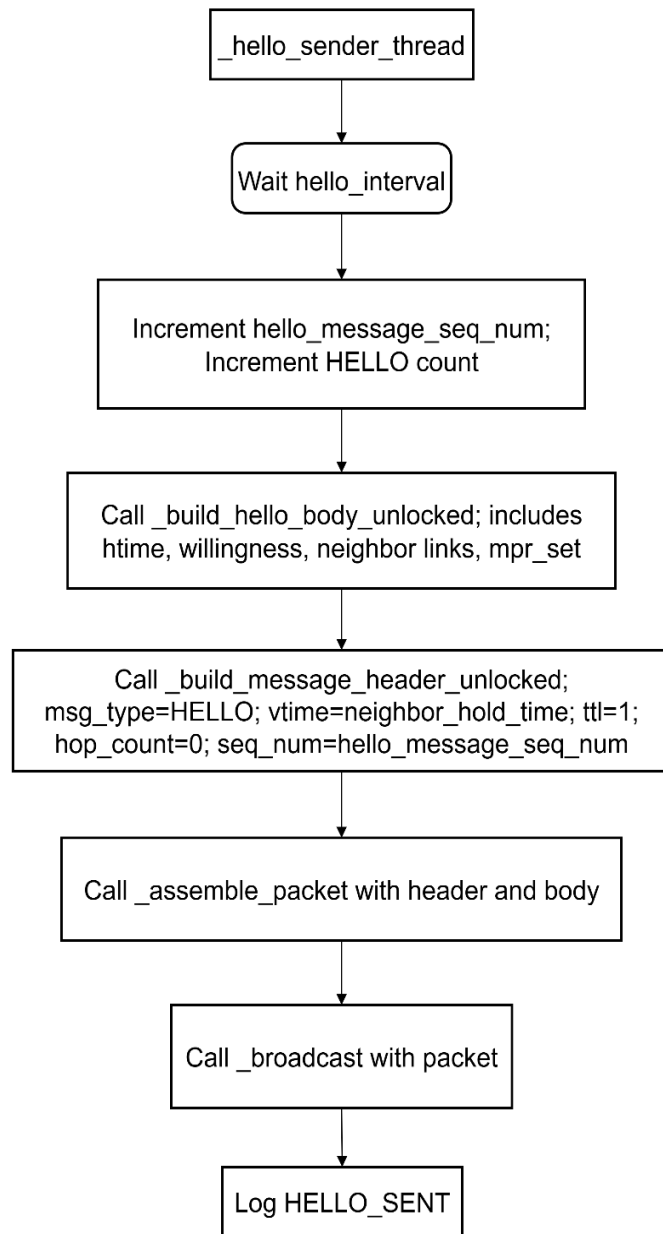


Figure B.6: Hello Sender Thread

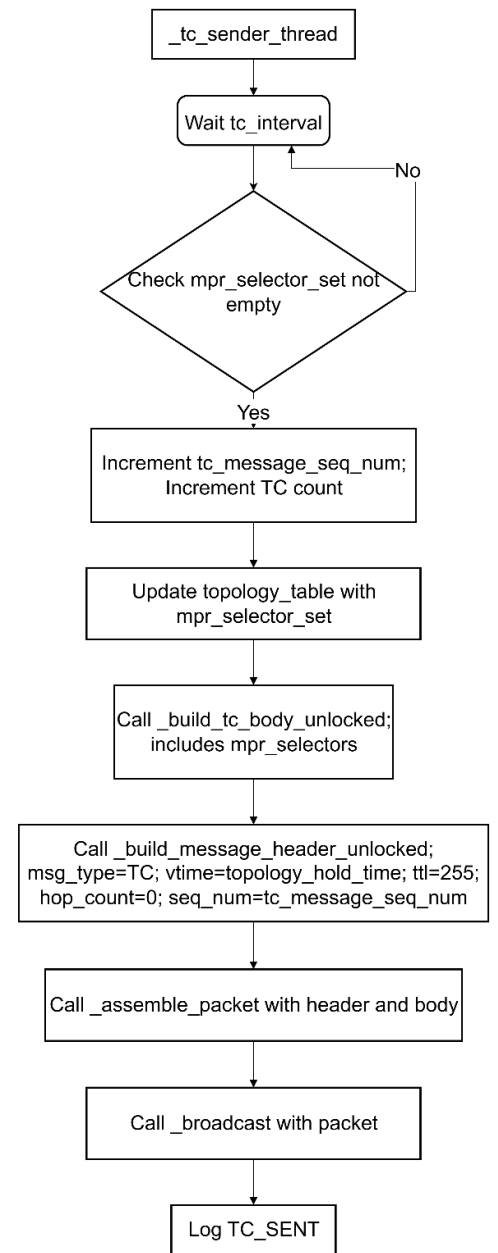


Figure B.6: TC Sender Thread

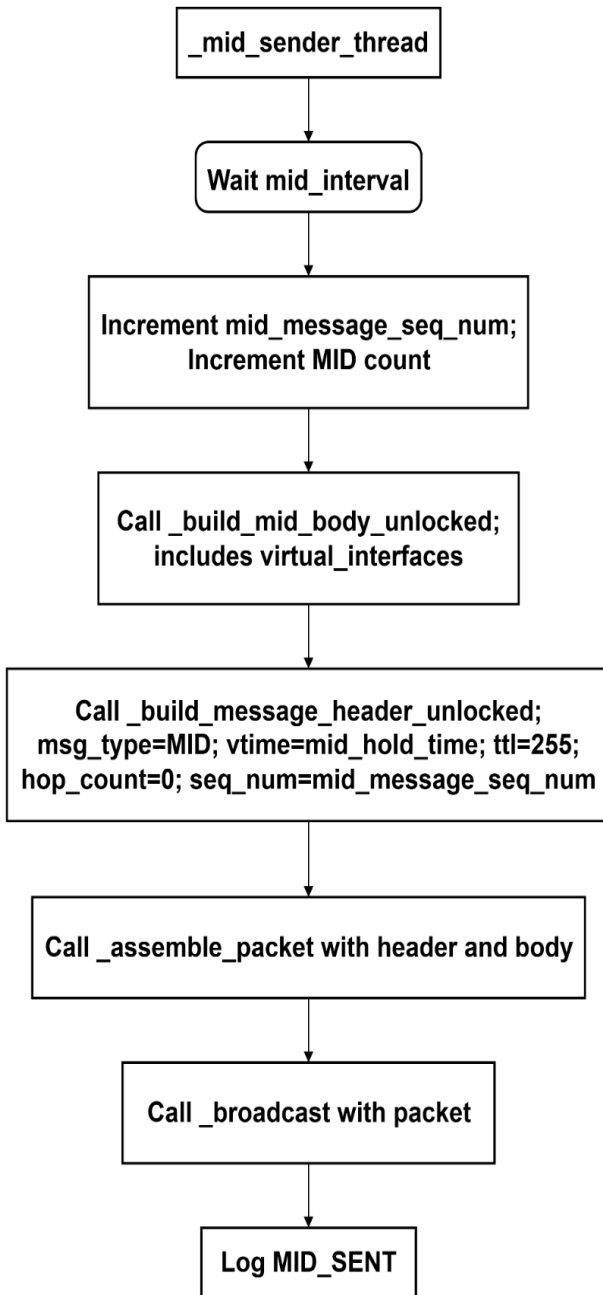


Figure B.8: MID Sender Thread

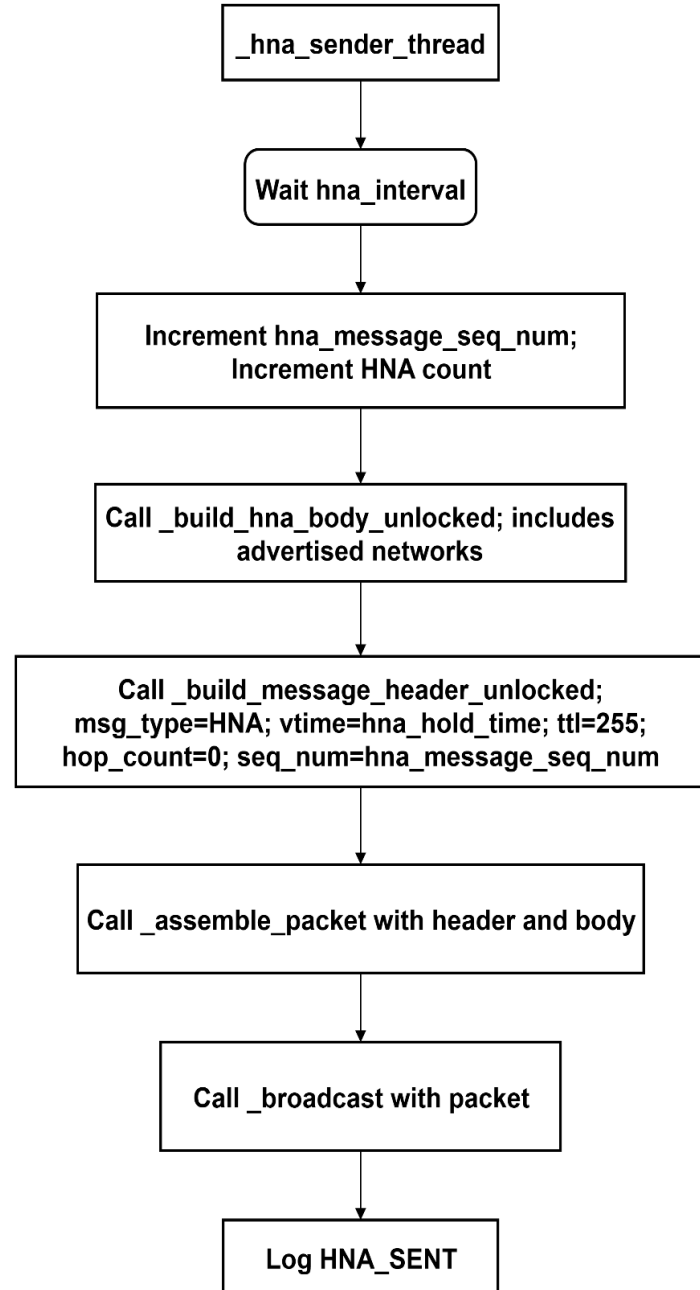


Figure B.9: HNA Sender Thread

B.7 Implementation of the Receiver Thread and Dispatch Logic

This section provides the detailed implementation of the `_receiver_thread`, which serves as the central processing pipeline for all incoming network traffic. It complements the high-level design described in Section 3.37 by detailing the specific code constructs and logic for each stage of packet validation and dispatching. While Section 3.7 presents the conceptual architecture, Figure

B.10 provides the implementation-level flowchart showing the exact technical details, including socket configuration parameters, function call signatures, and exception handling procedures.

B.7.1 Socket Initialization and Main Loop

The thread begins by initializing a UDP socket, configured to listen on all network interfaces (0.0.0.0) on the designated OLSR port. The `SO_REUSEADDR` socket option is enabled to facilitate rapid restarts of the node during testing. The core of the thread is an infinite while True loop, which blocks on `receiver_sock.recvfrom()`, waiting to receive data.

```
def _receiver_thread(self):
    receiver_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    receiver_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    receiver_sock.bind(('0.0.0.0', self.port))

    while True:
        try:
            data, addr = receiver_sock.recvfrom(2048)
            # ... subsequent processing happens within this loop ...
```

B.7.2 Stage 1: Structural Validation (Decoding and Parsing)

The first operation within the loop is to attempt decoding and parsing the raw data. This entire block is wrapped in a `try...except` clause to handle structurally invalid data.

- **Implementation:** The raw data (a byte stream) is first decoded into a string using `data.decode('utf-8')`, and then parsed into a Python dictionary using `json.loads()`. If either of these operations fails (e.g., due to non-UTF-8 characters or malformed JSON), a `json.JSONDecodeError`, `UnicodeDecodeError`, `KeyError`, or `TypeError` is caught. The except block contains only a pass statement,

meaning the corrupted packet is silently discarded, and the loop continues to the next iteration, ensuring high resilience.

```
# (Inside the while loop)
try:
    packet = json.loads(data.decode('utf-8'))
    # ...
except (json.JSONDecodeError, KeyError, TypeError):
    # Silently discard any packet that is not valid, well-formed JSON
    pass
```

B.7.3 Stage 2 & 3: Priority Command Processing and Blacklist

Validation

After successful parsing, the thread checks for special command messages before applying the standard OLSR isolation rules.

- **Priority Command Check:** The code first checks if the packet dictionary contains a "type" key with the value "COMMAND". This check is performed before the blacklist validation to ensure that external control messages can always reach the node, even if the sending machine is blacklisted for OLSR traffic. If it is a command, it is dispatched to `_process_command_message()`, and the `continue` statement skips the rest of the loop.
- **Blacklist Validation (Framework Extension):** If the packet is not a command, the sender's IP address (`sender_ip`) is checked against the `self.my_blacklist`. If a match is found, the `continue` statement discards the packet and skips to the next iteration. This enforces the experimental topology control.

```
# (Inside the try block, after parsing)
sender_ip = addr[0]

# Stage 2: Priority Command Processing
if packet.get("type") == "COMMAND":
    self._process_command_message(packet)
```

```

        continue

# Stage 3: Blacklist Validation
if sender_ip in self.my_blacklist:
    continue

```

B.7.4 Stage 4: Message Disassembly and Dispatch

For a valid OLSR packet that has passed all initial checks, the thread proceeds to process the individual messages it contains.

- **Message Iteration:** The code iterates through the list of messages found under the "messages" key.
- **Originator Validation:** For each message, it performs a final sanity check, extracting the `originator_address`. If the originator is the node itself (`self.my_ip`) or if the field is missing, the message is skipped to prevent self-processing loops.
- **Type-Based Dispatch:** Based on the `msg_type` string in the message header, a series of `if/elif` statements dispatches the message's header and body to the appropriate specialized processing function (e.g., `_process_hello_message`, `_process_tc_message`, etc.). This represents the final step in the receiver's pipeline, handing off the message for stateful processing.

```

# (Inside the try block, after blacklist validation)
for message in packet.get("messages", []):
    header = message.get("message_header", {})
    body = message.get("message_body", {})
    originator_ip = header.get("originator_address")

    if not originator_ip or originator_ip == self.my_ip:
        continue

    msg_type = header.get("msg_type")
    if msg_type == "HELLO":

```

```
        self._process_hello_message(header, body)
    elif msg_type == "TC":
        self._process_tc_message(header, body, sender_ip)
    # ... (elif for HNA, MID, DATA) ...
```

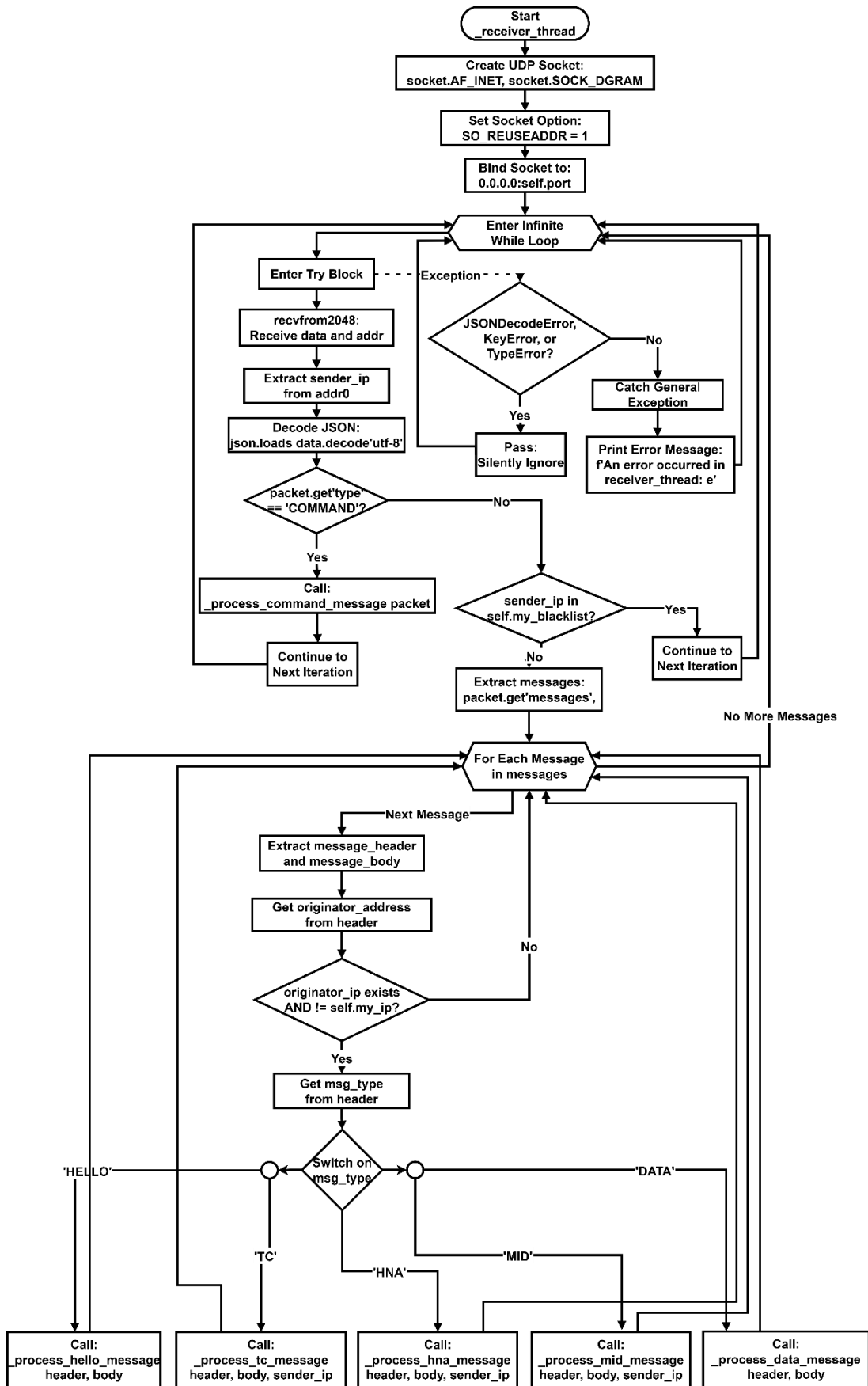


Figure B.10: Technical Implementation Details of the Receiver Thread

B.8 Implementation of Message Processing Functions

This section provides the detailed, step-by-step implementation logic for each of the specialized message processing functions mentioned in Section. These functions are the core of the protocol's state machine, responsible for interpreting incoming messages and updating the node's information bases. All functions operate under the protection of `self.table_lock` to ensure thread-safe modifications to shared data structures.

B.8.1 HELLO Message Processing (`_process_hello_message`)

This function is invoked upon receiving a HELLO message and is fundamental for maintaining local network awareness. Its logic is visually detailed in the flowchart in Figure B.11

- Purpose: To update the 1-hop and 2-hop neighbor tables and the MPR selector set based on information from direct neighbors.
- Implementation: The function executes a series of updates based on the content of the HELLO message body:
 1. Log Reception: It first logs the reception of the message.
 2. Determine Link Status: It inspects the links field in the message body to see if the current node's IP is being advertised by the sender. If so, the link is marked as symmetric (SYM); otherwise, it is asymmetric (ASYM).
 3. Update MPR Selector Set: It checks if the link is symmetric and if the current node's IP is in the sender's advertised `mpr_set`. If both are true, the originator is added

to `self.mpr_selector_set`; otherwise, it is removed. This step is how a node learns it has been chosen as an MPR.

4. Update 2-Hop Neighbor Table: It iterates through the neighbors advertised by the sender. Any neighbor that is not the node itself and not already a direct (1-hop) neighbor is added to the `self.two_hop_table`.

```
def _process_hello_message(self, header, body):
    with self.table_lock:
        originator_ip = header['originator_address']
        self.general_logger.info(f"HELLO_RECEIVED,from={originator_ip}")

        # Determine link status
        advertised_ips = {link.get('neighbor_ip') for link in
body.get('links', [])}
        link_status = 'SYM' if self.my_ip in advertised_ips else 'ASYM'

        # Update 1-hop neighbor table
        self.neighbor_table[originator_ip] = {'status': link_status,
'last_heard': time.time()}

        # Check if neighbor discovery is complete (for initialization)
        if not self._neighbor_discovery_complete:
            sym_neighbors = {ip for ip, data in
self.neighbor_table.items() if data['status'] == 'SYM'}
            if len(sym_neighbors) >= self.expected_sym_neighbors:
                self.general_logger.info("NEIGHBOR_DISCOVERY_COMPLETE")
                self._neighbor_discovery_complete = True

        # Update MPR selector set
        advertised_mpr_set = set(body.get('mpr_set', []))
        if link_status == 'SYM' and self.my_ip in advertised_mpr_set:
            self.mpr_selector_set.add(originator_ip)
        else:
            self.mpr_selector_set.discard(originator_ip)

        # Update 2-hop neighbor table
        for candidate_ip in advertised_ips:
            if candidate_ip and candidate_ip != self.my_ip and
candidate_ip not in self.neighbor_table:
                self.two_hop_table[candidate_ip] = {'via_neighbor':
originator_ip, 'last_updated': time.time()}
```

```
        self._pretty_print_tables(f"Tables updated after HELLO from  
{originator_ip}")
```

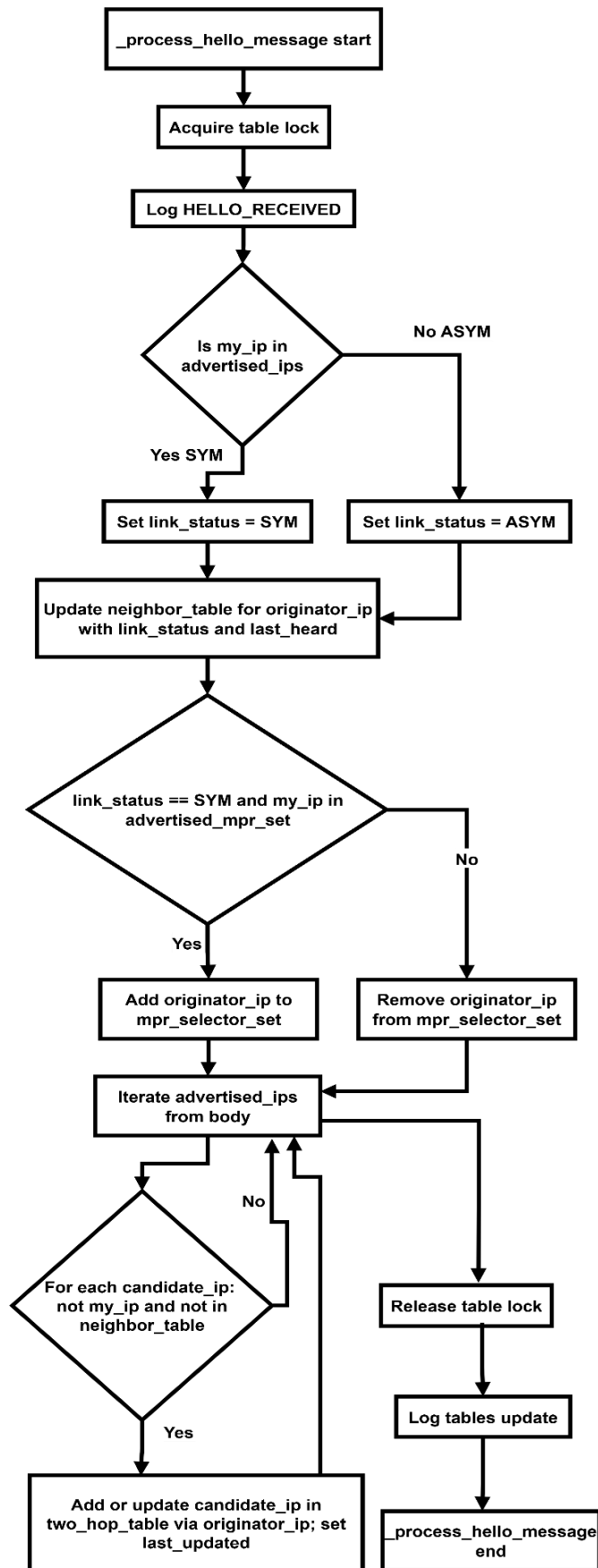


Figure B.11: Process Hello Message

B.8.2 TC Message Processing (`_process_tc_message`)

This This function processes TC messages, which are essential for building the network-wide topology graph. The processing logic for TC, HNA, and MID messages follows a unified design, as illustrated in the high-level flowchart in Figure A.6. This unified model shows the common steps such as sequence number validation and MPR-based forwarding. The specific, detailed implementation flow for a TC message is then presented in Figure B.12.

- Purpose: To update the `topology_table` with advertised MPR selector links and to forward the TC message if required.
- Implementation:
 1. Duplicate Suppression: It first performs a sequence number check. If the message is stale or a duplicate, it is discarded.
 2. Clear Old Entries: It removes all previous topology entries originated by this specific originator. This is crucial to ensure the topology information remains fresh.
 3. Add New Entries: It iterates through the `mpr_selectors` advertised in the message body and adds new tuples to the `self.topology_table`, recording the link between the selector and the originator (the MPR).
 4. Forwarding Logic: It checks if the message's TTL is greater than 1 and if the sender is in the node's `mpr_selector_set`. If both conditions are met, it decrements the `ttn` and rebroadcasts the message.

```
def _process_tc_message(self, header, body, sender_ip):  
    with self.table_lock:  
        originator = header['originator_address']
```

```

        seq_num = header['msg_seq_num']
        self.general_logger.info(f"TC_RECEIVED,originator={originator},seq={seq_num},from={sender_ip}")

        if self.last_tc_seq_nums.get(originator, -1) >= seq_num:
            return
        self.last_tc_seq_nums[originator] = seq_num
        current_time = time.time()

        # Clear old topology information from this originator
        for t_tuple in [t for t in self.topology_table if t[1] == originator]:
            del self.topology_table[t_tuple]

        # Add new topology information
        for selector_ip in body.get("mpr_selectors", []):
            self.topology_table[(selector_ip, originator)] = {'seq_num': seq_num, 'last_updated': current_time}

        self.general_logger.info(f"TOPOLOGY_TABLE_UPDATED,originator={originator}")
        self._pretty_print_tables(f"Tables updated after TC from {originator}")

        # Forwarding logic
        if header['ttl'] > 1 and sender_ip in self.mpr_selector_set:
            header['ttl'] -= 1
            forward_packet = {"messages": [{"message_header": header, "message_body": body}]}
            forward_packet["packet_header"] = self._build_packet_header_unlocked()
            self._broadcast(json.dumps(forward_packet).encode('utf-8'))
            self.general_logger.info(f"TC_FORWARDED,originator={originator},seq={seq_num}")

```

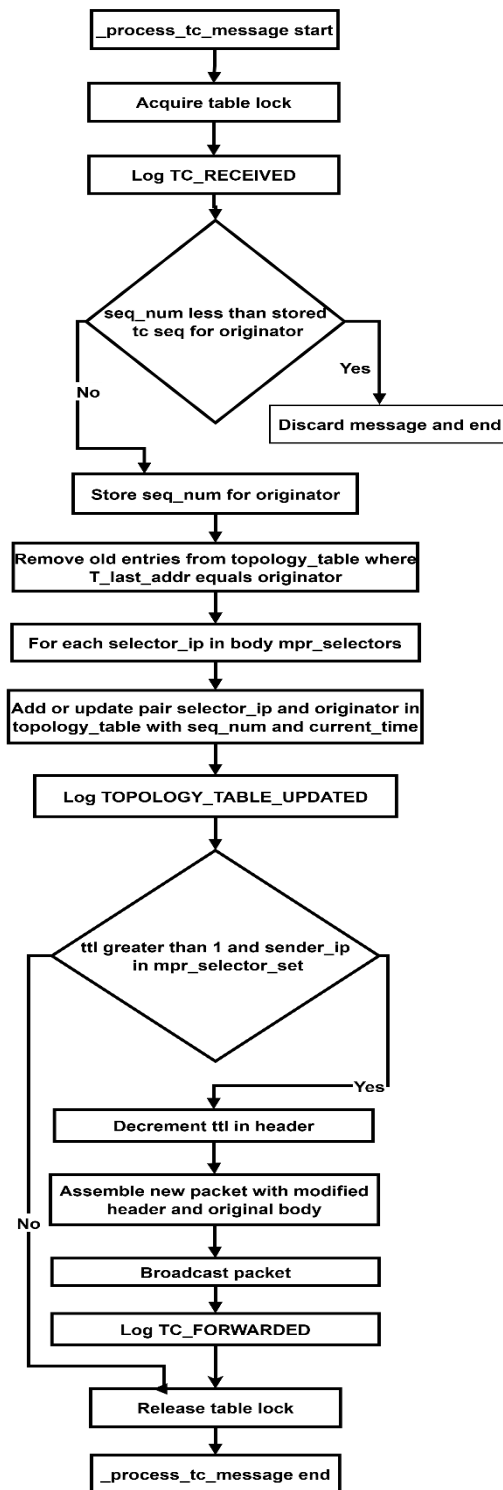


Figure B.12 Detailed Implementation Flow of TC Message Processing

B.8.3 HNA and MID Message Processing

These functions handle auxiliary messages (HNA and MID) and follow the same unified logic as TC message processing, as shown in the general flowchart

in Figure A.6. They share the same core mechanisms for sequence number checking and MPR-based forwarding. The specific implementation details for HNA and MID messages are then provided in their respective detailed flowcharts, Figure B.13 and Figure B.14.

- `_process_hna_message`: After passing a sequence number check, this function iterates through the advertised networks in the message body and updates the `self.hna_table`, mapping each external network to its gateway (the originator). It then applies the same MPR-based forwarding logic.
- `_process_mid_message`: After its sequence number check, this function updates the `self.interface_association_table`, mapping each advertised secondary interface to its main address (the originator). It also shares the identical forwarding logic.

```
def _process_hna_message(self, header, body, sender_ip):
    # ... (Implementation is analogous to _process_tc_message, but
    # updates hna_table)

def _process_mid_message(self, header, body, sender_ip):
    # ... (Implementation is analogous to _process_tc_message, but
    # updates interface_association_table)
    (Note: Full code for these two functions is omitted for brevity as their
    structure is identical to _process_tc_message, with only the target
    table changing.)
```

B.8.4 DATA Message Processing (`_process_data_message`)

This function is a key part of the framework's custom data plane and is not part of the standard OLSR control plane. Its logic is shown in the flowchart in Figure B.15.

- Purpose: To handle the reception and forwarding of application-level data packets.

- Implementation:

1. Loop Detection: It first checks if the node's own IP is already in the `path_taken` list. If so, the packet is dropped.
2. Update Path: The node's IP is appended to the `path_taken` list.
3. Destination Check: It checks if the `final_destination` is the current node.
 - If the receiver is the Destination: The packet has arrived. The function calculates the end-to-end delay, logs the RECEIVED event with all relevant metrics (including the full path), and increments the `DATA_RECEIVED` counter.
 - If Not Destination (Forwarding): The function queries `self.routing_table` for a route to the `final_destination`. If a route exists, it retrieves the `next_hop`, updates the packet's hop count, re-assembles the packet, and sends it directly to the `next_hop`. The `DATA_FORWARDED` counter is incremented. If no route is found, the packet is dropped.

```
def _process_data_message(self, header, body):
    with self.table_lock:
        final_destination = body.get("final_destination")
        packet_id = body.get("packet_id")
        path = body.get("path_taken", [])

        # 1. Loop Detection
        if self.my_ip in path:
            print(f"LOOP DETECTED for packet {packet_id}. Path: {path}. Dropping.")
            return

        # 2. Update Path
        path.append(self.my_ip)
```

```

# 3. Destination Check
if final_destination == self.my_ip:
    # Packet has reached its destination
    self.data_traffic_counts['DATA_RECEIVED'] += 1
    creation_time = body.get("creation_timestamp", 0)
    delay = time.time() - creation_time
    path_str = "->".join(path)

    self.data_logger.info(f"RECEIVED,{packet_id},{time.time()}',{
header['originator_address']},{self.my_ip},{delay:.4f},{path_str}")
    print(f" Received DATA packet {packet_id} from
{header['originator_address']}. Delay: {delay:.4f}s. Path: {path_str}")

else:
    # Forwarding Logic
    route_info = self.routing_table.get(final_destination)
    if route_info:
        next_hop = route_info['next_hop']
        header['hop_count'] += 1
        body['path_taken'] = path

        packet = self._assemble_packet(header, body)

        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as
sock:
            sock.sendto(json.dumps(packet).encode('utf-8'),
(next_hop, self.port))

            self.data_traffic_counts['DATA_FORWARDED'] += 1
    else:
        print(f"No route to {final_destination}, dropping DATA
packet {packet_id}")

```

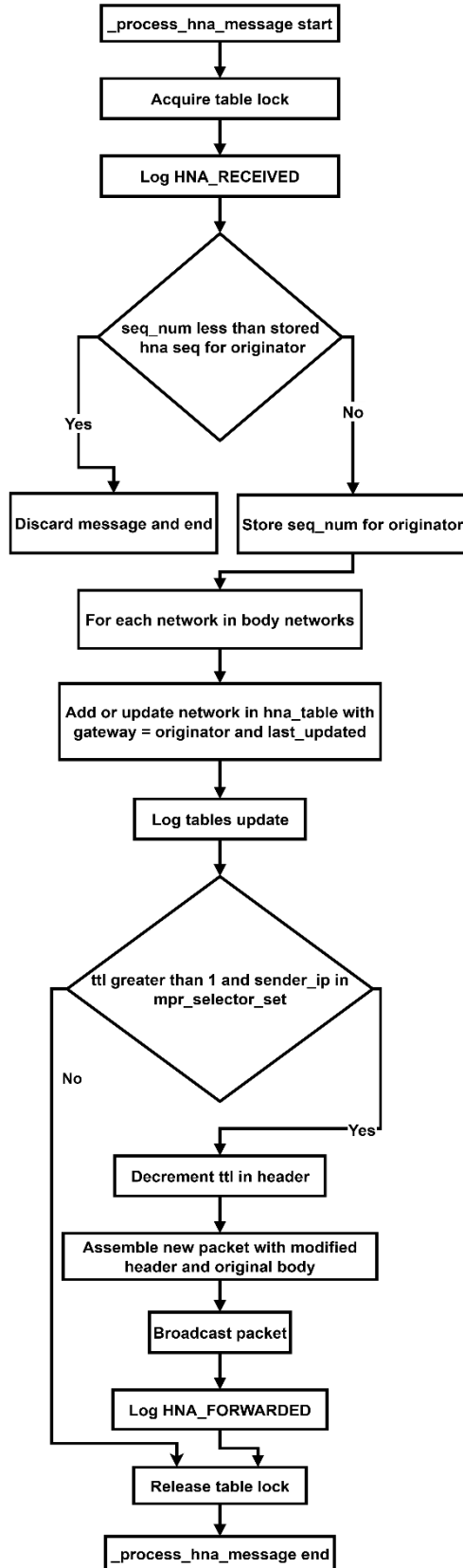
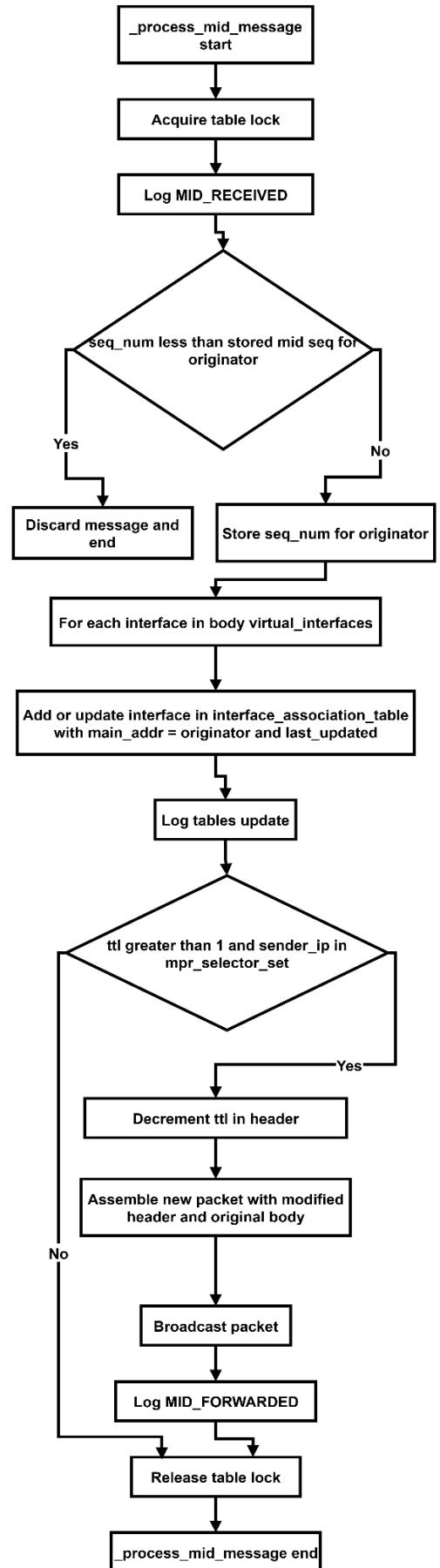


Figure B.13 :Detailed Implementation Flow of HNA Message Processing



FigureB.14:Detailed Implementation Flow of MID Message Processing

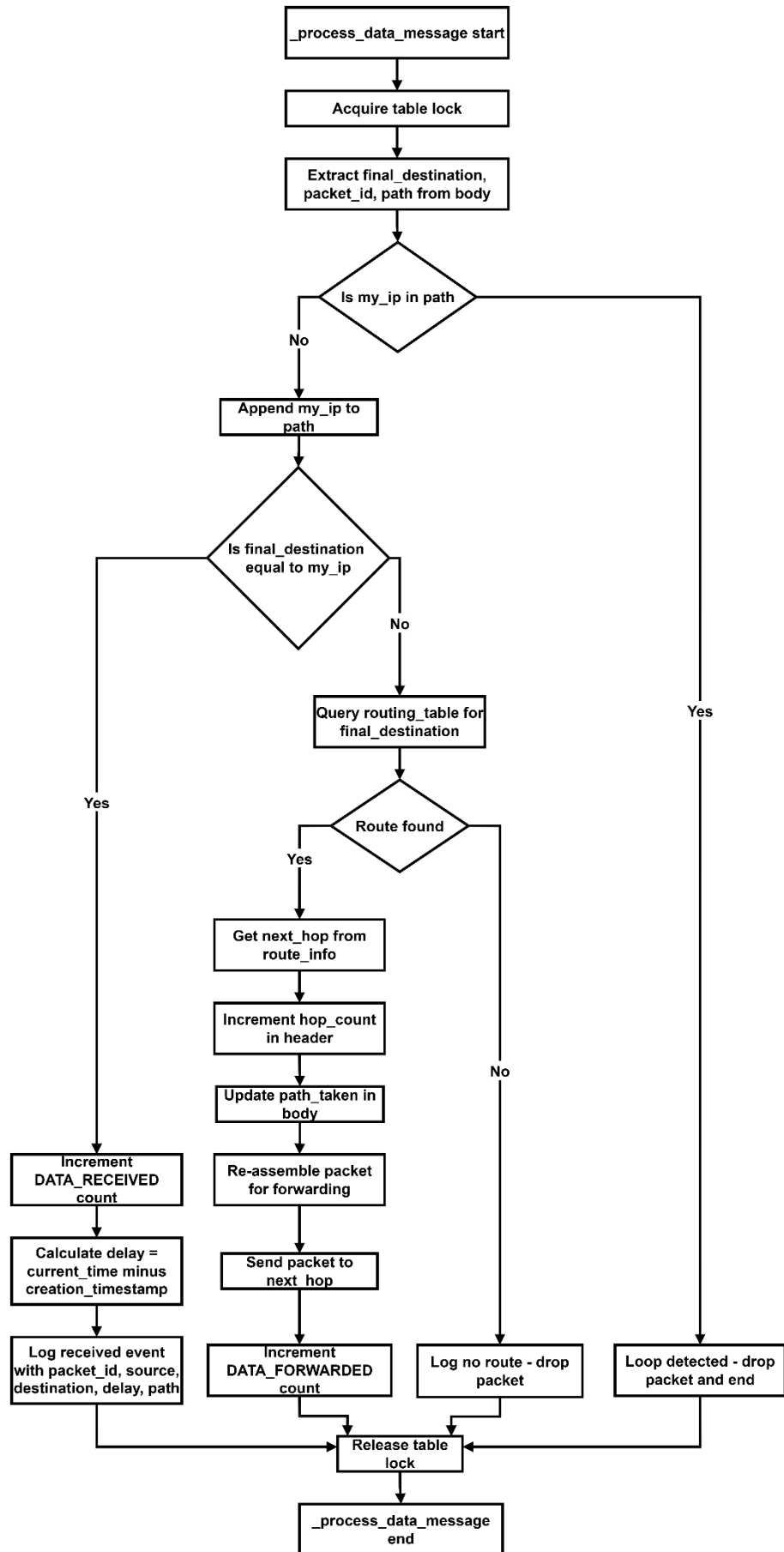


Figure B.15: Detailed Implementation Flow of DATA Message Processing

B.8.5 COMMAND Message Processing (`_process_command_message`)

This function provides an external control interface that operates independently of the standard OLSR protocol logic. Unlike OLSR control messages, command messages bypass the blacklist filtering mechanism, enabling external systems to issue control directives to the node. Its logic is shown in the flowchart in Figure B.16.

- Purpose

To process external command messages and dispatch appropriate handler threads for executing control operations without interfering with real-time OLSR protocol processing.

- Implementation

1. Parameter Extraction

The function begins by extracting the command parameters from the incoming command message structure:

```
params = command.get("params", {})  
dest = params.get("destination")  
count = params.get("count")
```

This step retrieves the nested `params` dictionary and extracts the specific parameters required for command execution. If the `params` key does not exist, an empty dictionary is used as the default to prevent `KeyError` exceptions.

2. Command Type Verification

The function checks whether the received command is of type `START_PDR_TEST`:

```
if command.get("command") == "START_PDR_TEST" and dest and count:
```

This conditional statement performs two critical validations:

- **Command Type Check:** Verifies that the command type matches the expected `START_PDR_TEST` string
- **Parameter Validation:** Ensures that both required parameters (`dest` and `count`) are present and not `None`

If either condition fails, the function returns immediately without taking any action, effectively ignoring unrecognized or incomplete commands.

3. Command Acknowledgment

Upon successful validation, the function prints a confirmation message to the console:

```
print(f"Received command to send {count} packets to {dest}")
```

This provides immediate feedback for debugging and monitoring purposes, confirming that the command has been received and accepted for processing.

4. Asynchronous Task Execution

The function spawns a new thread to execute the data transmission task:

```
threading.Thread(target=self._send_data_task, args=(dest, count)).start()
```

This design decision is critical for maintaining system responsiveness. By delegating the actual packet transmission to a separate thread, the function ensures that:

- The receiver thread is not blocked by time-consuming operations

- OLSR protocol messages continue to be processed without delay
- Multiple commands can be executed concurrently if needed

The spawned thread executes the `_send_data_task` method, which iteratively sends the specified number of data packets to the destination address.

Key Design Features

1. Independence from OLSR Protocol

Command messages are processed before the blacklist filtering is applied in the `_receiver_thread`. This architectural decision ensures that external control operations remain functional even when OLSR-level isolation rules are in effect. The relevant code in `_receiver_thread` shows this priority:

```
if packet.get("type") == "COMMAND":
    self._process_command_message(packet)
    continue

# OLSR blacklist filtering is applied only after command check
if sender_ip in self.my_blacklist:
    continue
```

2.Thread Safety

Although this function does not directly modify shared data structures (and thus does not acquire `table_lock`), the spawned `_send_data_task` thread will acquire the lock when accessing the routing table and other shared resources. This ensures thread-safe access to node state during packet transmission.

3.Extensibility

The current implementation specifically handles the `START_PDR_TEST` command, but the design can be easily extended to support additional command types by adding more conditional branches:

```
if packet.get("type") == "COMMAND":
    self._process_command_message(packet)
    continue

# OLSR blacklist filtering is applied only after command check
if sender_ip in self.my_blacklist:
```

continue

This extensible architecture allows the framework to support diverse control operations without modifying the core OLSR implementation.

Use Cases:

1. Performance Evaluation

The `START_PDR_TEST` command enables controlled Packet Delivery Ratio (PDR) testing by triggering the transmission of a specified number of data packets to a target destination. This is essential for evaluating the framework's data plane performance under various network conditions.

2. Automated Testing

External testing scripts can issue commands to multiple nodes simultaneously, coordinating complex multi-node experiments without requiring manual intervention or code modifications.

3. Runtime Control

The command interface provides a mechanism for runtime control and reconfiguration, enabling dynamic adjustment of node behavior during operation.

Complete Function Code:

```
def _process_command_message(self, command):
    params = command.get("params", {})
    dest = params.get("destination")
    count = params.get("count")

    if command.get("command") == "START_PDR_TEST" and dest and count:
        print(f"Received command to send {count} packets to {dest}")
        threading.Thread(target=self._send_data_task, args=(dest,
count)).start()
```

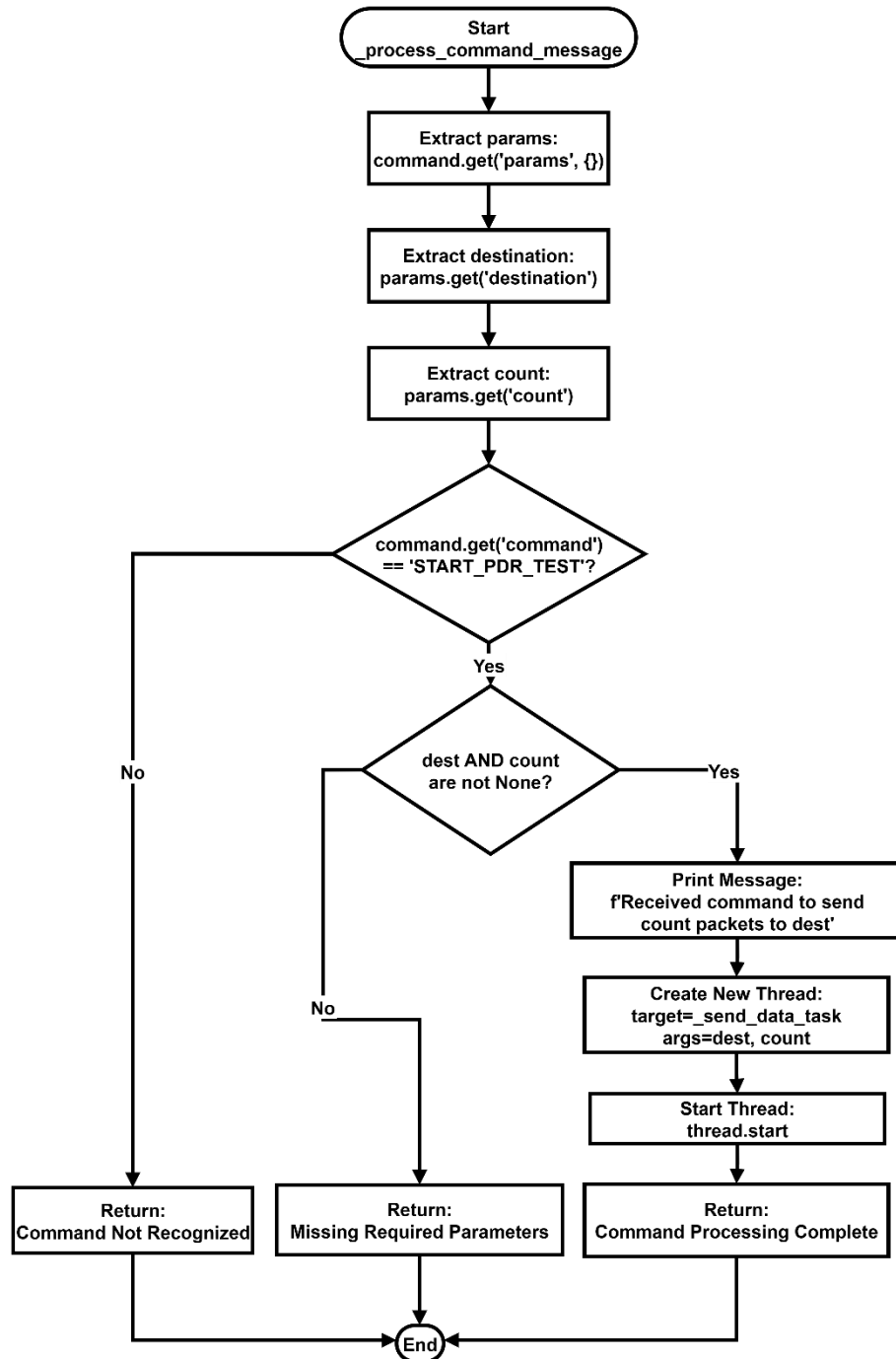


Figure B.16: Detailed Implementation Flow of Command Message Processing

B.9 Implementation of Routing Table Calculation

This section provides the detailed implementation of the routing table calculation logic, complementing the design overview in Section 3.3.9. It

covers both the periodic thread that triggers the calculation and the core function that executes the multi-phase computation process.

B.9.1 The Routing Table Thread (`_routing_table_thread`)

This daemon thread orchestrates the periodic recalculation of the routing table, ensuring route freshness.

- Purpose: To periodically and safely invoke the core routing calculation function.
- Implementation: The thread first pauses for a brief initial period (`self.tc_interval + 0.5`) upon startup. This delay allows the node to receive at least one round of TC messages from its neighbors, ensuring the first calculation is based on some initial topology data. Afterwards, it enters an infinite loop, sleeping for `self.routing_table_interval`. In each iteration, it acquires the `table_lock` to guarantee exclusive access to the information bases during computation, calls `_calculate_routing_table()`, and logs the event.

```
def _routing_table_thread(self):
    # Initial delay to allow for topology discovery
    time.sleep(self.tc_interval + 0.5)

    while True:
        with self.table_lock:
            self._calculate_routing_table()
            self.general_logger.info("ROUTING_TABLE_RECALCULATED")

        time.sleep(self.routing_table_interval)
```

B.9.2 The Core Calculation Function (`_calculate_routing_table`)

This function contains the complete logic for generating the routing table, from graph construction to path integration. The entire process is executed

atomically under the protection of the `table_lock`. The conceptual design of this multi-phase process is visually detailed in the flowchart in Figure B.17, while the implementation lifecycle within the `_routing_table_thread` is detailed in Figure B.18.

Phase 1: Graph Construction

- **Implementation:** A directed graph is constructed in memory as a dictionary.
1. **Node (Vertex) Aggregation:** A set of all known nodes (`all_nodes`) is created by aggregating IPs from all relevant information bases: `self.my_ip`, `self.neighbor_table`, `self.topology_table`, and the gateways in `self.hna_table`.
 2. **Edge Creation:** Edges are added to the graph:
 - **Symmetric 1-Hop Links:** For every symmetric neighbor in `self.neighbor_table`, bidirectional edges are added between the current node and the neighbor.
 - **Topology (MPR) Links:** For every tuple in `self.topology_table`, a directed edge is added from the MPR (last) to its selector (dest).

```
# Snippet from _calculate_routing_table
graph = {}
all_nodes = {self.my_ip} | set(self.neighbor_table.keys()) | \
             {t[0] for t in self.topology_table} | {t[1] for t in
self.topology_table}
for gateway_data in self.hna_table.values():
    all_nodes.add(gateway_data['gateway'])

for node in all_nodes:
    graph[node] = set()

# Add edges for symmetric neighbors
for neighbor, data in self.neighbor_table.items():
    if data['status'] == 'SYM':
```

```

graph[self.my_ip].add(neighbor)
graph[neighbor].add(self.my_ip)

# Add edges from topology table (MPR links)
for (dest, last) in self.topology_table:
    if dest in graph and last in graph:
        graph[last].add(dest)

```

Phase 2: Shortest Path Computation (Dijkstra's Algorithm)

- **Implementation:** The code implements Dijkstra's algorithm to find the shortest path from `self.my_ip` to all other nodes.
 1. **Initialization:** A `distances` dictionary is initialized with infinity for all nodes, and a `previous_nodes` dictionary (to reconstruct paths) is initialized to `None`. The distance to `self.my_ip` is set to 0. A priority queue (`pq`) is initialized with the starting node.
 2. **Iteration:** The algorithm iterates while the priority queue is not empty, always processing the node with the smallest known distance. For each neighbor of the current node, if a shorter path is found, the `distances` and `previous_nodes` dictionaries are updated, and the neighbor is pushed to the priority queue.

```

# Snippet from _calculate_routing_table
distances = {node: float('infinity') for node in all_nodes}
previous_nodes = {node: None for node in all_nodes}
distances[self.my_ip] = 0
pq = [(0, self.my_ip)]

while pq:
    dist, node = heapq.heappop(pq)
    if dist > distances[node]:
        continue
    for neighbor in graph.get(node, set()):
        if distances[node] + 1 < distances[neighbor]:
            distances[neighbor] = distances[node] + 1
            previous_nodes[neighbor] = node
            heapq.heappush(pq, (distances[neighbor], neighbor))

```

Phase 3: Route Translation and Integration

- Implementation: The results from Dijkstra's algorithm are translated into the final routing table structure.
 1. Path Backtracking: The code iterates through all reachable destinations. For each destination, it backtracks through the `previous_nodes` chain until it finds the node that is a direct neighbor of `self.my_ip`. This neighbor is the correct `next_hop`.
 2. HNA Route Integration: It then iterates through the `self.hna_table`. For each external network, it finds the route to its gateway. If a route exists, a new route to the external network is created with the same `next_hop` as the gateway, but with the distance incremented by one. A special case handles gateways that are direct symmetric neighbors.
 3. Atomic Update: Finally, the `self.routing_table` attribute is atomically replaced with the `new_routing_table`.

```
# Snippet from _calculate_routing_table
new_routing_table = {}
for dest_node, dist in distances.items():
    if dest_node == self.my_ip or dist == float('infinity'):
        continue

    # Backtrack to find the next hop
    path_node = dest_node
    while path_node and previous_nodes.get(path_node) != self.my_ip:
        path_node = previous_nodes.get(path_node)

    if path_node:
        new_routing_table[dest_node] = {'next_hop': path_node,
                                         'distance': dist}

# Integrate HNA routes
for network, data in self.hna_table.items():
```

```

gateway_ip = data['gateway']
route_to_gateway = new_routing_table.get(gateway_ip)
if route_to_gateway:
    new_routing_table[network] = {'next_hop':
route_to_gateway['next_hop'], 'distance': route_to_gateway['distance'] +
1}
    elif gateway_ip in self.neighbor_table and
self.neighbor_table[gateway_ip]['status'] == 'SYM':
        new_routing_table[network] = {'next_hop': gateway_ip,
'distance': 2}

# Atomically update the main routing table
self.routing_table = new_routing_table

```

B.10 Implementation of the State Cleanup Mechanism

This section provides the detailed implementation of the `_cleanup_thread`, the housekeeping process responsible for maintaining the freshness of all OLSR information bases, as introduced in Section 3.3.10. The operational flow is visually summarized in the flowchart in Figure B.19.

B.10.1 The Cleanup Thread (`_cleanup_thread`)

This function runs as a persistent daemon thread, periodically scanning all state tables and purging entries that have exceeded their designated validity times (hold times).

- Purpose: To prevent "state decay" by removing stale entries from all information bases, ensuring that routing decisions are based only on timely and valid network information.
- Implementation: The thread operates in an infinite loop, synchronized with the `hello_interval` to ensure regular checks.
 1. Acquire Lock: It begins each cycle by acquiring the `table_lock` to ensure that no other thread can access the tables during the cleanup process.

2. Identify Expired Entries: It constructs several "to remove" lists by iterating through each information base. For each entry, it compares the current time (`time.time()`) with the entry's last known timestamp (`last_heard` or `last_updated`) and its corresponding hold time (e.g., `self.neighbor_hold_time`).
 3. Perform Removal: If any of the "to remove" lists are not empty, it proceeds to delete the identified stale entries from their respective tables. This includes removing neighbors, 2-hop neighbors, topology tuples, HNA routes, and MID associations.
 4. Log and Print: After a successful cleanup operation, it logs a `CLEANUP_EVENT` and calls `_pretty_print_tables` to provide a snapshot of the updated tables for debugging purposes.
- Key Logic Point: The logic for removing 2-hop neighbors is particularly comprehensive. An entry is removed not only if it has timed out, but also if its `via_neighbor` is no longer a direct neighbor, or if the entry itself has been "promoted" to a 1-hop neighbor. This ensures the integrity and non-redundancy of the neighbor information bases. Similarly, when a topology tuple is removed, the corresponding entry in `last_tc_seq_nums` is also cleared to allow for future updates from that originator.

```
def _cleanup_thread(self):
    while True:
        time.sleep(self.hello_interval)
        with self.table_lock:
            current_time = time.time()

            # Identify stale entries from all tables
            neighbors_to_remove = [ip for ip, data in
self.neighbor_table.items() if current_time - data['last_heard'] >
self.neighbor_hold_time]
            two_hops_to_remove = [ip for ip, data in
list(self.two_hop_table.items()) if (ip in self.neighbor_table or
```

```

data['via_neighbor'] not in self.neighbor_table or current_time -
data['last_updated'] > self.two_hop_hold_time)]
    tuples_to_remove = [t_tuple for t_tuple, data in
self.topology_table.items() if current_time - data['last_updated'] >
self.topology_hold_time]
    hna_to_remove = [net for net, data in self.hna_table.items()
if current_time - data['last_updated'] > self.hna_hold_time]
    mid_to_remove = [ip for ip, data in
self.interface_association_table.items() if current_time -
data['last_updated'] > self.mid_hold_time]

    # If any stale entries were found, perform the removal
    if neighbors_to_remove or two_hops_to_remove or
tuples_to_remove or hna_to_remove or mid_to_remove:
        for ip in neighbors_to_remove:
            if ip in self.neighbor_table: del
self.neighbor_table[ip]
            self.mpr_selector_set.discard(ip)

        for ip in set(two_hops_to_remove):
            if ip in self.two_hop_table: del
self.two_hop_table[ip]

        for t_tuple in tuples_to_remove:
            if t_tuple in self.topology_table: del
self.topology_table[t_tuple]
            self.last_tc_seq_nums.pop(t_tuple[1], None)

        for net in hna_to_remove:
            if net in self.hna_table: del self.hna_table[net]

        for ip in mid_to_remove:
            if ip in self.interface_association_table: del
self.interface_association_table[ip]

    self.general_logger.info("CLEANUP_EVENT")
    self._pretty_print_tables(f"🔧 Cleanup Event")

```

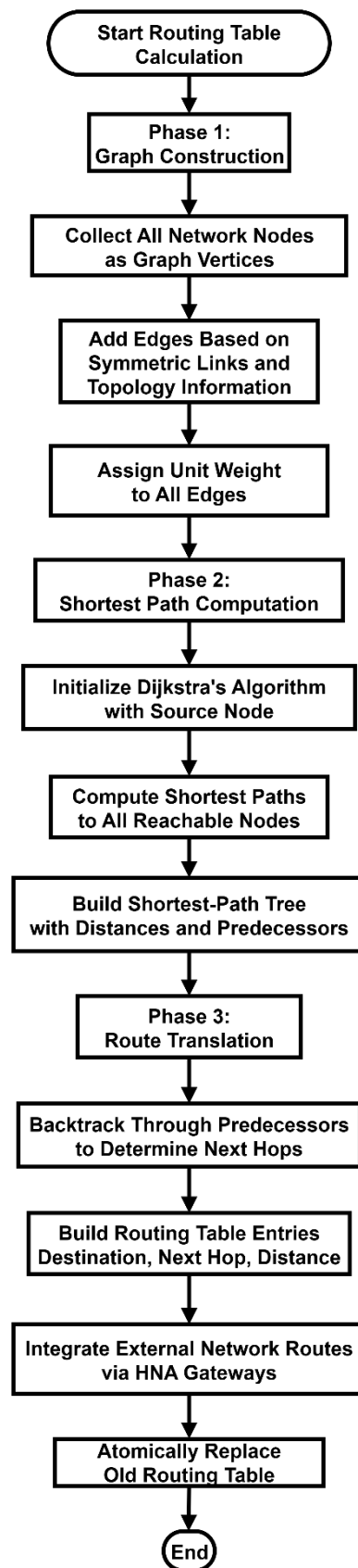


Figure B.17: Conceptual Design of Routing Table Calculation

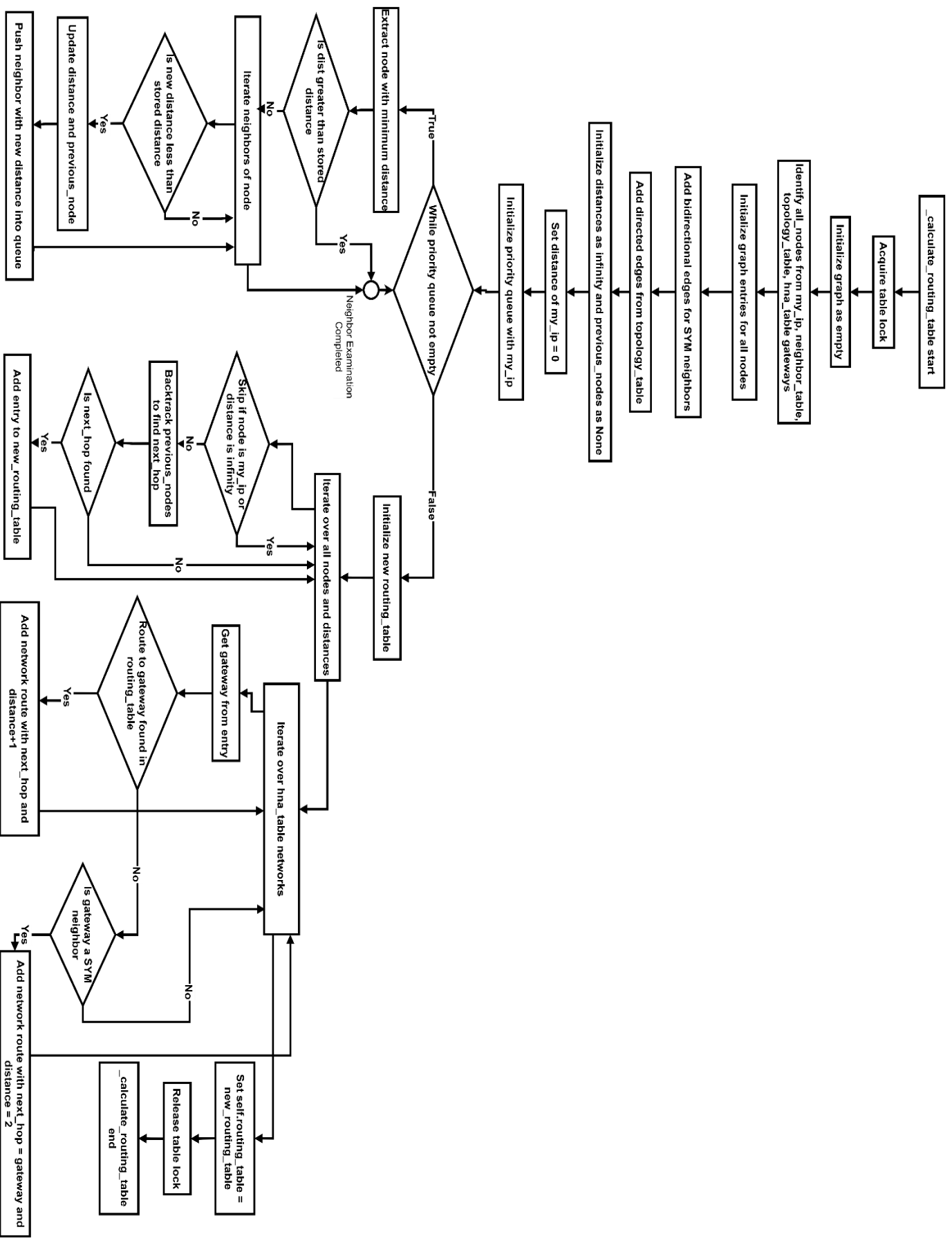


Figure B.18: Detailed Implementation Flow of Routing Table Calculation



Figure B.19: Detailed Implementation Flow of the Cleanup Thread

B.11 Implementation of the Logging Framework

This section provides the detailed implementation of the dual-logger architecture described in Section. This framework is a core component of the research instrumentation, providing structured data for both protocol verification and performance analysis. The setup logic is visually summarized in the flowchart in Figure B.20.

B.11.1 Logger Setup (`_setup_loggers`)

This function is called during the `OlsrNode`'s initialization (`__init__`) to configure and prepare the two distinct loggers.

- Purpose: To create, configure, and attach file handlers for the `general_logger` and `data_logger`, ensuring that all subsequent log messages are written to the correct files in the correct format.
- Implementation:
 1. General Logger: It retrieves a logger instance named `general_{self.my_ip}`. A `FileHandler` is created to write to `olsr_log_{self.my_ip}.txt`. A custom formatter is defined to include a high-precision timestamp (including milliseconds), which is critical for analyzing the sequence and timing of control plane events.
 2. Data Logger: It retrieves a separate logger instance named `data_{self.my_ip}`. A `FileHandler` is created to write to `pdr_results_{self.my_ip}.csv`. Crucially, after creating the handler, the function immediately writes a header row (`"EventType,PacketID,Timestamp,Source,Destination,Delay,Path"`) to the CSV file. This ensures the file is always

correctly formatted for direct use with data analysis libraries like Pandas.

```
def _setup_loggers(self):
    # --- General Logger for control plane events ---
    self.general_logger = logging.getLogger(f'general_{self.my_ip}')
    self.general_logger.setLevel(logging.INFO)
    if not self.general_logger.handlers:
        fh_general = logging.FileHandler(f'olsr_log_{self.my_ip}.txt',
mode='w')
        # Formatter includes milliseconds for precise event timing
        formatter = logging.Formatter('%(asctime)s.%(msecs)03d -
%(message)s', datefmt='%H:%M:%S')
        fh_general.setFormatter(formatter)
        self.general_logger.addHandler(fh_general)

    # --- Data Logger for performance metrics (CSV format) ---
    self.data_logger = logging.getLogger(f'data_{self.my_ip}')
    self.data_logger.setLevel(logging.INFO)
    if not self.data_logger.handlers:
        fh_data = logging.FileHandler(f'pdr_results_{self.my_ip}.csv',
mode='w')
        self.data_logger.addHandler(fh_data)

    # Write the CSV header immediately upon creation
    self.data_logger.info("EventType,PacketID,Timestamp,Source,Desti
nation,Delay,Path")
```

B.11.2 Usage Examples

The loggers are invoked throughout the `OlsrNode` code to record significant events.

- General Logger Usage (Event-based): Used to record control plane actions and state changes.

```
# Example from _hello_sender_thread
self.general_logger.info("HELLO_SENT")

# Example from _routing_table_thread
self.general_logger.info("ROUTING_TABLE_RECALCULATED")
```

Example of some entires of “olsr_log_192.168.10.1.txt” file

```
09:51:07.681 - NODE_START,ip=192.168.10.1
09:51:09.686 - HELLO_SENT
09:51:11.690 - HELLO_SENT
09:51:12.689 - MPR_SET_RECALCULATED,set=[ ]
09:51:12.694 - HNA_SENT
09:51:13.191 - ROUTING_TABLE_RECALCULATED
09:51:13.693 - HELLO_SENT
09:51:15.697 - HELLO_SENT
09:51:17.694 - MPR_SET_RECALCULATED,set=[ ]
09:51:17.701 - HELLO_SENT
09:51:17.702 - HNA_SENT
09:51:18.193 - ROUTING_TABLE_RECALCULATED
09:51:19.705 - HELLO_SENT
09:51:21.708 - HELLO_SENT
09:51:22.699 - MPR_SET_RECALCULATED,set=[ ]
09:51:22.709 - HNA_SENT
09:51:22.807 - HELLO_RECEIVED,from=192.168.10.2
```

- Data Logger Usage (Metric-based): Used exclusively for recording performance data related to application-level packets. The log format is strictly maintained to match the CSV header.

```
# Example from _send_data_packet (when a packet is first sent)
# Note the empty fields for Delay and Path, maintaining column
structure.
self.data_logger.info(f"SENT,{packet_id},{time.time()},{self.my_ip},{final_destination},,")

# Example from _process_data_message (when a packet is received)
# All fields are populated, including the calculated delay and the full
path.
self.data_logger.info(f"RECEIVED,{packet_id},{time.time()},{header['originator_address']},{self.my_ip},{delay:.4f},{path_str}")
```

Example of “pdr_results_192.168.10.5.csv” file :

	A	B	C	D	E	F	G
1	EventType	PacketID	Timestamp	Source	Destination	Delay	Path
2	RECEIVED	1	1758688459	192.168.10.3	192.168.10.5	0.0272	192.168.10.3->192.168.10.5
3	RECEIVED	2	1758688459	192.168.10.3	192.168.10.5	0.0269	192.168.10.3->192.168.10.5
4	RECEIVED	3	1758688460	192.168.10.3	192.168.10.5	0.027	192.168.10.3->192.168.10.5
5	RECEIVED	4	1758688460	192.168.10.3	192.168.10.5	0.027	192.168.10.3->192.168.10.5
6	RECEIVED	5	1758688461	192.168.10.3	192.168.10.5	0.0269	192.168.10.3->192.168.10.5
7	RECEIVED	6	1758688461	192.168.10.3	192.168.10.5	0.0281	192.168.10.3->192.168.10.5
8	RECEIVED	7	1758688462	192.168.10.3	192.168.10.5	0.0268	192.168.10.3->192.168.10.5
9	RECEIVED	8	1758688462	192.168.10.3	192.168.10.5	0.0285	192.168.10.3->192.168.10.5
10	RECEIVED	9	1758688463	192.168.10.3	192.168.10.5	0.0275	192.168.10.3->192.168.10.5

B.11.3 Shutdown Hook for Traffic Summaries

To capture a final summary of the node's workload, a `try...except KeyboardInterrupt` block is used in the main `start()` method. When the user stops the node (Ctrl+C), the `except` block is executed before exiting.

- Purpose: To log the total counts of all control and data traffic handled by the node during its runtime.
- Implementation: The `KeyboardInterrupt` handler logs the final state of the `self.control_traffic_counts` and `self.data_traffic_counts` dictionaries as JSON strings to the general log file.

```
# Snippet from start() method
try:
    while True: time.sleep(1)
except KeyboardInterrupt:
    # Log final traffic counts upon shutdown
    self.general_logger.info(f"CONTROL_TRAFFIC,{json.dumps(self.control_
traffic_counts)}")
    self.general_logger.info(f"DATA_TRAFFIC,{json.dumps(self.data_traffi
c_counts)}")
    print(f"\n[{self.my_ip}] Shutdown signal received. Exiting.")
```

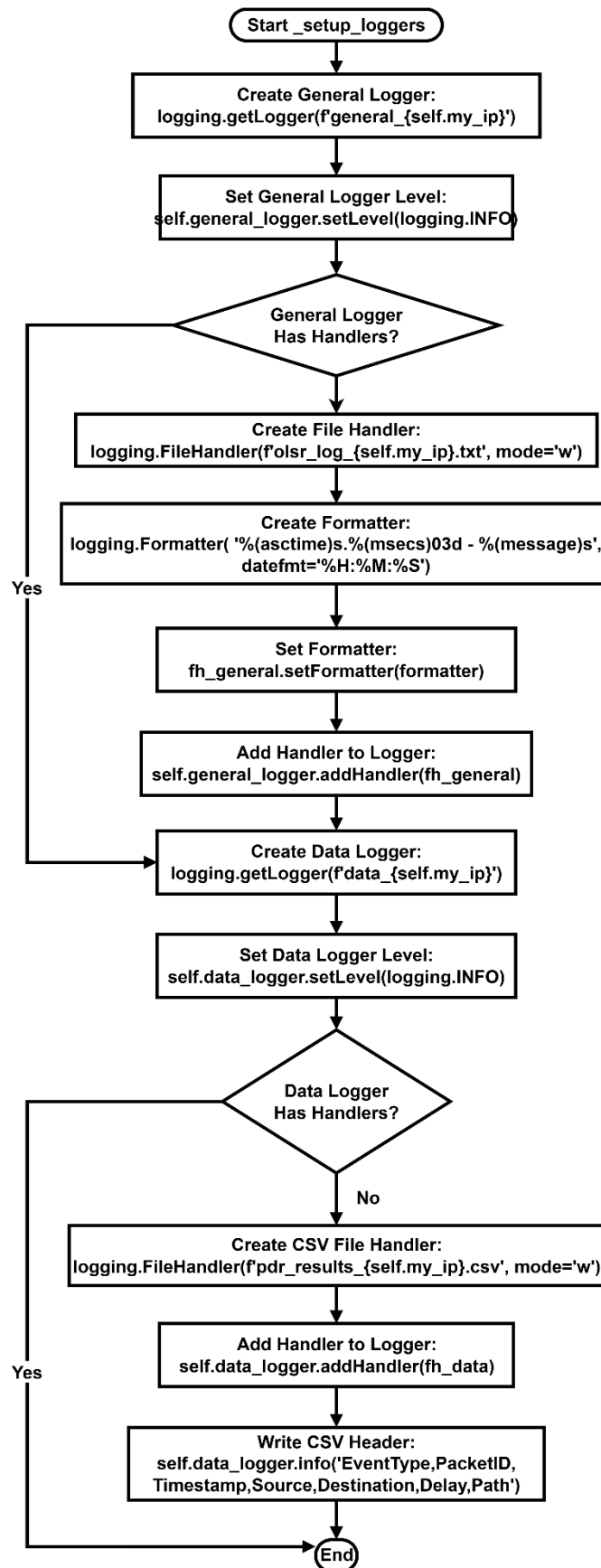


Figure B.20 Detailed Implementation Flow of Logging Infrastructure Setup

B.12 Implementation of the External Traffic Generator

This section provides the implementation details for the `test_runner.py` script, the external test orchestrator described in Section 3.6.3. This script acts as a command-line client, instructing a target `OlsrNode` to initiate a data transmission task. The operational flow is visually summarized in the flowchart in Figure B.21.

B.12.1 The `test_runner.py` Script

The script is a standalone Python program designed to be executed from the command line. It takes the source node, destination node, and number of packets as arguments.

- Purpose: To orchestrate performance tests by sending a standardized COMMAND message to a source `OlsrNode`, rather than injecting data directly into the network.
- Implementation:
 1. Argument Parsing: It uses `sys.argv` to parse command-line arguments for the source IP, destination IP, and the number of packets.
 2. Command Packet Construction: It constructs a Python dictionary (`command_packet`) with `type` set to "COMMAND" and `command` set to "START_PDR_TEST". The destination IP and packet count are nested within the `params` dictionary.
 3. Serialization and Transmission: The dictionary is serialized to a JSON string and encoded to UTF-8 bytes. A UDP socket is then used to send these bytes to the specified source node's IP address on the standard OLSR port (698).

4. User Feedback and Guidance: After sending the command, the script provides comprehensive feedback to the user. It confirms that the command was sent, calculates an estimated completion time for the test, and provides explicit instructions on which log files (`pdr_results_*.csv` and `olsr_log_*.txt`) to check on the nodes for analysis.

```
# test_runner.py
import socket
import json
import time
import sys

OLSR_PORT = 698

def run_test(source_ip, dest_ip, num_packets):
    """Sends a command to the source node to start a PDR test."""

    print(f"--- Starting PDR/Delay Test ---")
    print(f"Source Node:      {source_ip}")
    print(f"Destination Node: {dest_ip}")
    print(f"Packet Count:      {num_packets}")

    command_packet = {
        "type": "COMMAND",
        "command": "START_PDR_TEST",
        "params": {
            "destination": dest_ip,
            "count": num_packets
        }
    }

    command_bytes = json.dumps(command_packet).encode('utf-8')

    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.sendto(command_bytes, (source_ip, OLSR_PORT))

    print("\nTest command sent to the source node.")
    print("The test will run in the background on the OLSR network.")
    # Calculate and display estimated time for user convenience
    estimated_time = (num_packets * 0.5) + 10
```

```

    print(f"Wait for at least {estimated_time:.0f} seconds for the test
to complete.")
    print("After waiting, you can collect the log files from the nodes
for analysis.")
    print("Look for 'pdr_results_*.csv' and 'olsr_log_*.txt' on each
node.")

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Usage: python3 test_runner.py <source_ip>
<destination_ip> <num_packets>")
        print("Example: python3 test_runner.py 192.168.10.5 192.168.10.1
100")
        sys.exit(1)

    source_node = sys.argv[1]
    destination_node = sys.argv[2]
    number_of_packets = int(sys.argv[3])

    run_test(source_node, destination_node, number_of_packets)

```

B.12.2 Interaction with OlsrNode

The functionality of the `test_runner.py` script is contingent on the `OlsrNode`'s ability to interpret the `COMMAND` message. This interaction occurs in two stages within the node.

Stage 1: Command Processing (`_process_command_message`)

- **Implementation:** This function is called by the `_receiver_thread` when a packet of type `COMMAND` is identified. It inspects the command field. If it matches `"START_PDR_TEST"`, it extracts the destination and count from the `params` dictionary and starts a new daemon thread, targeting the `_send_data_task` function. This ensures that the data transmission task runs in the background without blocking the main receiver thread.

```

# In OlsrNode class
def _process_command_message(self, command):
    params = command.get("params", {})

```

```

dest = params.get("destination")
count = params.get("count")

if command.get("command") == "START_PDR_TEST" and dest and count:
    print(f"Received command to send {count} packets to {dest}")
    # Start the data sending task in a new thread
    threading.Thread(target=self._send_data_task, args=(dest,
count)).start()

```

Stage 2: Data Transmission Task (`_send_data_task` and `_send_data_packet`)

- **Implementation:** The `_send_data_task` function, now running within the `OlsrNode`, iterates the specified number of times (`count`). In each iteration, it calls `_send_data_packet`, which is responsible for the actual data generation and transmission. This function performs a route lookup in `self.routing_table`, constructs the DATA message, and sends it to the identified `next_hop`. This confirms that the test traffic is subject to the node's own, current routing logic.

```

# In OlsrNode class
def _send_data_task(self, destination, count):
    for i in range(count):
        self._send_data_packet(destination, f"Test packet {i+1}", i + 1)
        # A small delay between packets to avoid flooding the network
        time.sleep(0.5)

def _send_data_packet(self, final_destination, payload, packet_id):
    # ... (Full implementation as provided in the main source code) ...

```

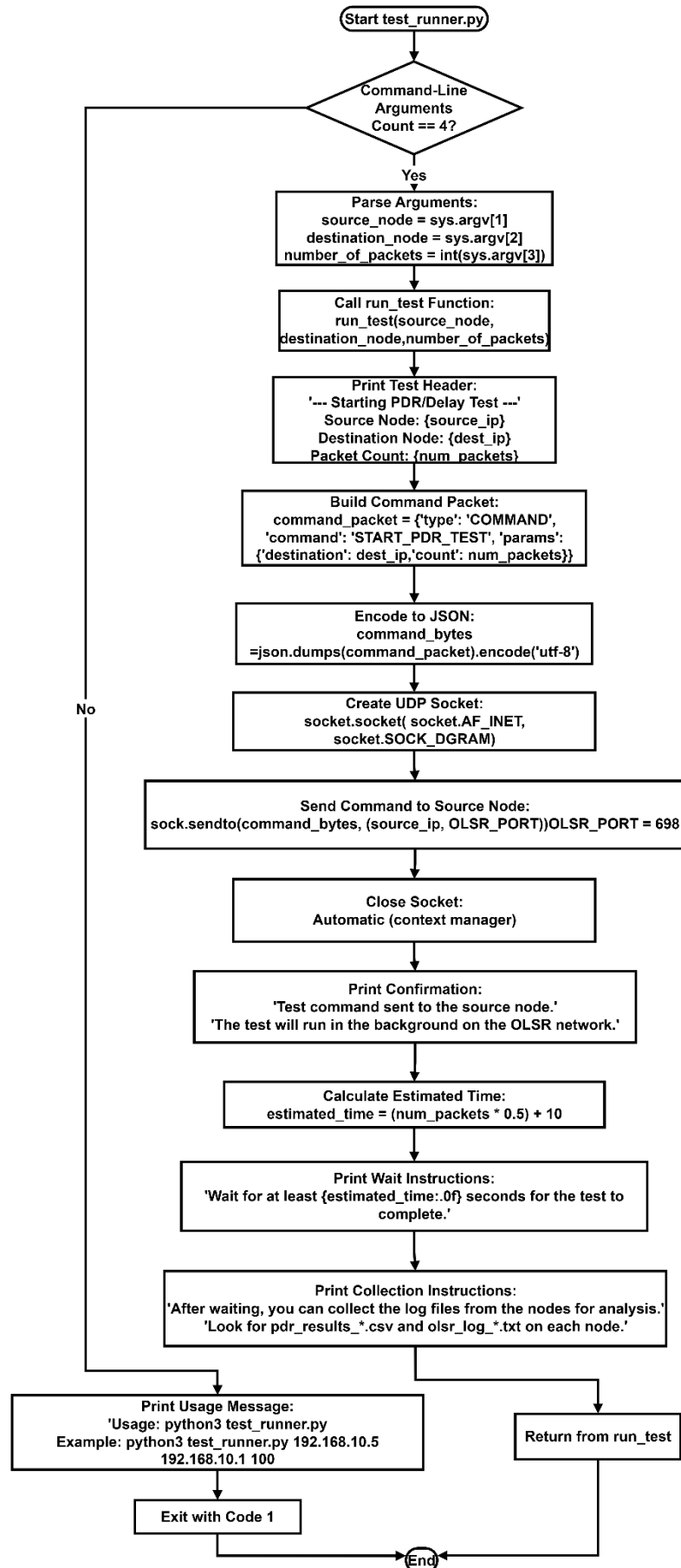


Figure B.21: Detailed Implementation Flow of the External Traffic Generator Script

Appendix C Virtual Transport Layer Implementation Details

The VirtualUDP class implements a standardized, connectionless, best-effort transport protocol in user-space, serving as the essential link between the application layer and the Virtual Network Layer (VNL). This implementation focuses on replicating the core UDP functionalities of virtual port management, multiplexing, demultiplexing, and data integrity checking using Python constructs.

C.1 VirtualUDP Class: The Technical Blueprint

This section presents the final technical blueprint for the VirtualUDP class, which embodies the canonical User Datagram Protocol (UDP) service model within the Virtual Transport Layer (VTL). This implementation is designed to provide a standardized, best-effort transport service, focusing on the critical mechanisms of Multiplexing and Demultiplexing through virtual ports, and incorporating a checksum for data integrity checking. This design step is crucial for ensuring the VTL aligns with the standardization of the underlying Network Layer (OLSR).

To provide an in-depth view of the software architecture, the reference begins with a Class Diagram (Figure C.1), illustrating the primary components of VirtualUDP and its required association with the underlying OlsrNode (VNL). This is followed by two detailed tables: Table C.1 covers the essential, state-managing attributes used for port registration and receive queuing, and Table C.2 summarizes the operational methods and their roles in transmission and demultiplexing logic.

<<Virtual Transport Layer>> VirtualUDP	
+ network_layer : OlsrNode	
- bound_ports : Dict	
- receive_queues : Dict	
+ __init__(network_layer)	
+ bind(port)	
+ sendto(dest_ip, dest_port, payload, source_port=None)	
+ recvfrom(port)	
- _receive_from_network(source_ip, network_payload)	

Figure C. 1: Architectural Overview of the VirtualUDP Class

Table C. 1 Key Attributes of the VirtualUDP Class

Attribute Name	Type	Description
network_layer	OlsrNode object	Reference to the underlying Virtual Network Layer, used to delegate data transmission.
bound_ports	Dictionary	Stores active virtual ports that applications are listening on. Used during demultiplexing to determine if an incoming packet has a listener.
receive_queues	Dictionary of deque	A collection of limited-size (max 100) data queues, indexed by the virtual port number. This buffers incoming data until consumed by the application.

Table C. 2 Summary of Key VirtualUDP Methods

Method	Category	Role and Function
<code>__init__</code>	Initialization	Sets up the internal state and registers <code>_receive_from_network</code> as the callback function with the VNL.
<code>bind(port)</code>	Public API/Mux	Registers a virtual port as active and initializes a dedicated deque for incoming data on that port.
<code>sendto()</code>	Public API/Transport	Constructs the virtual UDP header (including source/destination ports and calculated checksum), encapsulates the application data, and delegates transmission to the VNL.
<code>recvfrom(port)</code>	Public API/Transport	Retrieves and removes the oldest item from the specified port's receive queue in a non-blocking manner.
<code>_receive_from_network()</code>	Internal/Demux	The VNL callback. Deserializes the packet, validates the checksum, and performs demultiplexing by

delivering the payload to the correct queue.

C.2 Implementation Details of VirtualUDP Core Functions

C.2.1 Constructor Method (`__init__`)

The constructor establishes the foundational link between the transport and network layers and initializes the data structures necessary for port management.

Python Implementation:

```
def __init__(self, network_layer):
    self.network_layer = network_layer
    # Register our receiving function with the network layer
    self.network_layer.set_data_callback(self._receive_from_network)
    # A dictionary to map port numbers to application callback functions
    self.bound_ports = {}
    # A dictionary of queues, one for each bound port, to hold incoming
    data
    self.receive_queues = {}
    print(f"[{self.network_layer.my_ip}] Virtual Transport Layer (UDP)
    initialized.")
```

Detailed Explanation:

The function performs two mandatory steps to prepare the layer for operation:

1. **Layer Binding:** It stores the `network_layer` reference and immediately registers its internal handling function, `self._receive_from_network`, using the VNL's `set_data_callback` interface. This is the inverse control mechanism that allows the VNL to push incoming packets up the stack.
2. **State Initialization:** It initializes `self.bound_ports` and `self.receive_queues`, which will serve as the runtime tables for managing virtual ports and buffering data.

C.2.2 Binding Method (bind)

The bind method is called by the application to signal that it is ready to receive data on a specific virtual port, thereby setting up the infrastructure for demultiplexing.

Python Implementation:

```
from collections import deque
# ... inside the VirtualUDP class ...
def bind(self, port, callback=None):
    """Allows an application to 'listen' on a virtual port."""
    if port not in self.bound_ports:
        # Create the queue for this port
        self.receive_queues[port] = deque(maxlen=100)
        # Actually register the port as bound.
        self.bound_ports[port] = callback if callback is not None else
True
        print(f"[Transport Layer] Port {port} is now bound.")
```

Detailed Explanation:

The function ensures that if the provided port is not already active, a new buffer is created and registered.

1. Queue Creation: A collections.deque object is instantiated for the new port, with a maximum size of 100 messages (maxlen=100).
2. Registration: The port is added to the self.bound_ports dictionary, effectively enabling demultiplexing for any incoming packets addressed to this port.

C.2.3 Sending Method (sendto)

The sendto method is the core transmission interface, responsible for adding the virtual UDP header and checksum before passing the packet down to the VNL. This method realizes the connectionless, datagram-oriented nature of the protocol.

Python Implementation:

```
import json
import zlib
import random
# ... inside the VirtualUDP class ...
def sendto(self, dest_ip, dest_port, payload, source_port=None):
    """Public API for the Application to send data."""
    if source_port is None:
        source_port = random.randint(49152, 65535) # Use an ephemeral
port
    payload_bytes = payload.encode('utf-8')

    # Create the virtual UDP header
    udp_header = {
        "src_port": source_port,
        "dst_port": dest_port,
        "length": 8 + len(payload_bytes), # 8 bytes for a typical UDP
header
        "checksum": zlib.crc32(payload_bytes)
    }

    # The payload for the network layer is the combination of our header
and the app data
    transport_payload = {
        "udp_header": udp_header,
        "data": payload
    }

    # Send the payload down to the network layer
    self.network_layer.send(dest_ip, json.dumps(transport_payload))
```

Detailed Explanation:

1. Port Assignment: If no `source_port` is provided by the application, a transient ephemeral port is randomly selected from the reserved range (49152 to 65535).
2. Checksum Calculation: To fulfill the standard service model, the VTL calculates a checksum (`zlib.crc32`) based on the encoded application payload bytes. This value is critical for integrity checking at the receiver.

3. Encapsulation: The method constructs a `udp_header` dictionary containing the source port, destination port, packet length, and the calculated checksum. This header is combined with the application data (`data`) into a final `transport_payload`.

4. Delegation: The complete `transport_payload` is serialized to a JSON string and sent to the specified `dest_ip` via the VNL's public send API (`self.network_layer.send`).

C.2.4 Receiving and Demultiplexing Method (`_receive_from_network`)

This callback function is the central receiving logic, responsible for deserialization, validating data integrity, and directing the packet to the correct application queue.

Python Implementation:

```
def _receive_from_network(self, source_ip, network_payload):
    """Callback function that the Network layer calls with new data."""
    try:
        transport_packet = json.loads(network_payload)
        header = transport_packet.get("udp_header", {})
        data = transport_packet.get("data", "")
        dest_port = header.get("dst_port")

        # Check if any application is listening on this port
        if dest_port in self.bound_ports:
            # Verify checksum
            checksum_received = header.get("checksum")
            checksum_calculated = zlib.crc32(data.encode('utf-8'))

            if checksum_received == checksum_calculated:
                # Checksum is valid. Deliver the data to the correct
                queue.
                source_port = header.get("src_port")
                self.receive_queues[dest_port].append(((source_ip,
                source_port), data))
            else:
                print(f"[Transport Layer] Checksum mismatch! Dropping
                packet from {source_ip}.")

        except (json.JSONDecodeError, TypeError):
```

```
# Ignore malformed transport packets
pass
```

Detailed Explanation:

1. Deserialization: The function attempts to parse the `network_payload` (JSON string) into a structured `transport_packet` dictionary.

2. Demultiplexing Check: It extracts the `dest_port` from the header and checks if it is present in `self.bound_ports`. If the port is not bound, the packet is silently discarded.

3. Integrity Check (Checksum): If the port is bound, the function calculates the checksum of the received data and compares it against the `checksum_received` from the header.

- If the checksums do not match, the packet is discarded, ensuring that corrupted data does not reach the application layer.

4. Delivery: If the integrity check passes, the data is delivered to the correct `self.receive_queues[dest_port]`, along with the source IP and source port, completing the demultiplexing process.

C.2.5 Data Retrieval Method (`recvfrom`)

This function provides the non-blocking API for the application layer to pull data from its bound port's queue.

Python Implementation:

```
def recvfrom(self, port, max_size=1024):
    """Public API for the Application to receive data from a bound
    port."""
    if port in self.receive_queues and self.receive_queues[port]:
        # Retrieve the oldest message from the queue for this port
        return self.receive_queues[port].popleft()
```

```
return None, None # Return None if no data is available
```

Detailed Explanation:

The function checks if the specified port is active and if its corresponding queue (`self.receive_queues[port]`) contains data. It uses the `popleft()` method of the deque to retrieve the oldest packet. Crucially, the function is non-blocking, returning `None, None` immediately if no data is available, allowing the application to execute other logic without interruption.

Appendix D Virtual Application Layer Implementation

Details

This appendix provides the granular technical details for the Application Layer implementation, `application.py`, which is designed to operate a client/server model utilizing the port-based services of the `VirtualUDP` class.

D.1 Architectural Blueprint of the Application Layer

To present a technical overview of this layer, this section commences with a UML Class Diagram illustrating the relationship between the application and the lower virtual layers. This is followed by Table D.1, detailing the essential configuration attributes, and Table D.2, which summarizes the primary operational tasks implemented within the application layer.

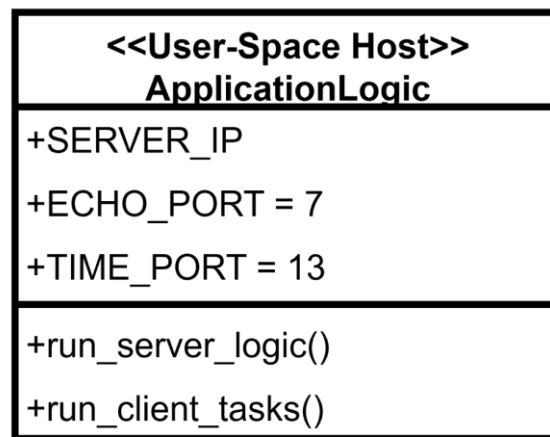


Figure D. 1 UML Class Diagram of the Application Layer Components

Table D. 1 Key Configuration Variables of the Application Script

Variable Name	Type	Purpose
MY_IP_ADDRESS	String	The IP address assigned to this specific node.
SERVER_IP	String	The designated IP address hosting the standard application services (Echo/Time).
ECHO_PORT	Integer (7)	The well-known virtual port number for the Echo service.
TIME_PORT	Integer (13)	The well-known virtual port number for the Time service.

Table D 2 Summary of Key Application Functions and Tasks

Method/Task	Category	Role and Function
if __name__ == "__main__"	Stack Orchestration	Initializes OlsrNode (VNL) and VirtualUDP (VTL) hierarchically, starts the OLSR background threads, and dispatches the appropriate application task based on the node role.
run_server_logic	Server Task	Binds to standard ports (ECHO_PORT, TIME_PORT). Continuously polls these ports using recvfrom(), processes the request (e.g., calculates time), and replies using sendto() to the client's source address/port.

run_client_tasks	Client Task	Selects and binds to a random ephemeral port for replies. Periodically uses sendto() to request services from the SERVER_IP and polls its bound port for incoming replies.
-------------------------	-------------	---

D.2 Implementation Details of Application Tasks

D.2.1 Initialization and Orchestration

The main execution block orchestrates the virtual stack startup, replacing the SimpleReliableTransport with the standardized VirtualUDP.

Python Implementation (Excerpt):

```
if __name__ == "__main__":
    am_i_server = (MY_IP_ADDRESS == SERVER_IP)

    # 1. Initialize the Network Layer (OLSR)
    network_layer = OlsrNode(my_ip=MY_IP_ADDRESS)

    # 2. Initialize the Transport Layer, giving it the network layer
    transport_layer = VirtualUDP(network_layer)

    # 3. Start the OLSR protocol in a background thread
    # ... (thread creation and start omitted for brevity) ...

    time.sleep(20) # Give OLSR time to build tables

    # 4. Start the Application task based on the node's role
    try:
        if am_i_server:
            run_server_logic(transport_layer)
        else:
            run_client_tasks(transport_layer)
    # ... (Exception handling omitted) ...
```

Detailed Explanation:

The main block performs the core virtualization steps: it instantiates the VNL (OlsrNode) and links it to the VTL by instantiating VirtualUDP(network_layer). The subsequent dispatch logic determines which application task (server or client) should run, passing the configured transport_layer object to the task for service consumption. A longer stabilization delay (20 seconds) is used here compared to the previous application to ensure the proactive OLSR protocol has fully converged across the multi-hop topology before transaction requests begin.

D.2.2 Server Logic (run_server_logic)

This task demonstrates the server-side use of port binding and demultiplexing, allowing the application to manage multiple standard services simultaneously.

Python Implementation (Excerpt):

```
def run_server_logic(transport_layer: VirtualUDP):
    """Binds to ports and listens for requests."""

    # Bind the application logic to specific virtual ports
    transport_layer.bind(ECHO_PORT)
    transport_layer.bind(TIME_PORT)

    while True:
        # Check the Echo port for incoming data
        (source_addr, data) = transport_layer.recvfrom(ECHO_PORT)
        if data:
            # Echo the data back to the sender's source port
            transport_layer.sendto(source_addr, source_addr[20], data)

        # Check the Time port for incoming data
        (source_addr, data) = transport_layer.recvfrom(TIME_PORT)
        if data:
            current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            # Send the current time back to the sender's source port
            transport_layer.sendto(source_addr, source_addr[20],
current_time)

            time.sleep(0.1) # Prevents the loop from consuming 100% CPU
```

Detailed Explanation:

1. Port Binding: The task explicitly calls `transport_layer.bind()` for both `ECHO_PORT` (7) and `TIME_PORT` (13). This registration enables the VTL's demultiplexing logic to correctly queue inbound packets destined for these ports.
2. Request Handling: The server uses the non-blocking `recvfrom(port)` to poll both service ports sequentially within the main loop.
3. Reply Generation: Upon successful data reception, the server extracts the client's full address (`source_addr`, which includes the IP and the ephemeral source port). The server then uses `sendto()` to reply, ensuring the response is directed back to the specific client port that originated the request.

D.2.3 Client Logic (`run_client_tasks`)

This task demonstrates the client-side use of datagram services, highlighting the selection of a source port and the use of multi-hop routing to reach the server.

Python Implementation (Excerpt):

```
def run_client_tasks(transport_layer: VirtualUDP):
    """Runs on client nodes. Periodically sends requests to the server
    in a loop."""

    # Choose a random ephemeral port for receiving replies.
    client_port = random.randint(49152, 65535)
    transport_layer.bind(client_port)

    # ... (Wait for network stabilization) ...

    while True:
        # --- Test 1: Echo Service ---
        message_to_echo = f"Hello from {MY_IP_ADDRESS}, packet
#{packet_counter}"
        transport_layer.sendto(SERVER_IP, ECHO_PORT, message_to_echo,
source_port=client_port)

        # --- Test 2: Time Service ---
```

```

        transport_layer.sendto(SERVER_IP, TIME_PORT, "time_request",
source_port=client_port)

    # --- Wait for replies ---
    time.sleep(3)

    while True:
        (source_addr, data) = transport_layer.recvfrom(client_port)
        if data:
            # Process reply
            # ...
        else:
            break
    time.sleep(5)

```

Detailed Explanation:

1. Ephemeral Port Assignment: The client randomly selects and binds a high-numbered port (`client_port`) for its source address. This bound port is where all replies from the server are expected to arrive.
2. Request Transmission: The client uses the `sendto()` interface, explicitly specifying the fixed `SERVER_IP` and the well-known service ports (`ECHO_PORT` or `TIME_PORT`). Critically, the client also includes its own `client_port` as the source port, enabling the server to address its replies correctly.
3. Reply Retrieval: After sending requests, the client waits and then repeatedly calls `recvfrom(client_port)` to check its bound queue for replies that have been demultiplexed by the VTL. This continuous polling validates the end-to-end communication flow, relying on the VNL's OLSR routing to deliver the packets across multiple hops.